

Ligero

Lightweight Sublinear Arguments Without a Trusted Setup

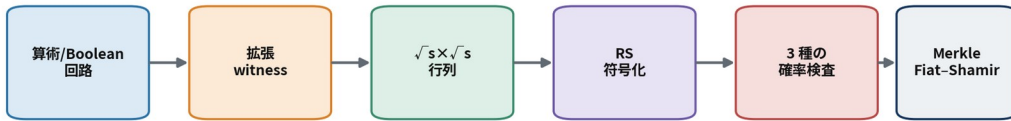
軽量・透明・平方根通信量の
汎用ゼロ知識 argument

原論文著者

Scott Ames / Carmit Hazay / Yuval Ishai / Muthuramakrishnan Venkitasubramaniam

対象：2022年 extended version (CCS 2017 論文の完全版)

Ligero の証明パイプライン



通信量 $\approx O(\sqrt{|C|})$ / 透明セットアップ / 対称鍵プリミティブのみ

中核：interleaved Reed-Solomon 符号と、ランダム線形結合＋少数列開示

本書の範囲

- 原論文本文・Appendix A-C の注釈付き網羅的翻訳
- interleaved Reed-Solomon 符号、線形・二次制約テスト、完全 ZK の数学
- MPC-in-the-head / ZKIPCP / Merkle / Fiat-Shamir の接続
- Aurora ・ ZK-STARK ・ Groth16 ・ PLONK ・ Spartan ・ Nova 系との比較
- 有限体 F_{17} の手計算例、実装・パラメータ設計・応用・監査観点

目次

前付

- 本書について
- エグゼクティブ・サマリー

Part I 背景と数学的準備

- 1 Ligero が位置する証明系の系譜
- 2 符号理論と多項式表現
- 3 IPCP、IOP、Merkle、Fiat-Shamir

Part II 原論文の注釈付き網羅訳

- 4 Abstract・Introduction・Our Results
- 5 Preliminaries の注釈付き翻訳
- 6 From MPC to ZKIPCP
- 7 A Direct ZKIPCP Construction
- 8 From ZKIPCP to ZK
- 9 Implementation and Experimental Results
- 10 Related Work and Open Problems
- 11 Conclusions の注釈付き翻訳
- 12 Appendix A–C の注釈付き翻訳

Part III 数学的仕組みの深掘り

- 13 なぜ平方根通信量になるのか
- 14 soundness 証明の解剖
- 15 完全零知識の数理
- 16 有限体 F_{17} の手計算例
- 17 parameter 設計の実務
- 18 ROM・Fiat-Shamir・実装安全性

Part IV Aurora・ZK-STARK・他の汎用 ZKP との比較

- 19 Ligero・Aurora・2018 ZK-STARK の共通基盤
- 20 三方式の強み・弱み
- 21 他の汎用 ZKP との比較
- 22 用途別の選択指針

Part V 応用・実装・監査

- 23 Ligero の応用
- 24 実装アーキテクチャ
- 25 security audit checklist

- 26 性能評価の設計
- 27 批判的評価
- 28 研究ロードマップ
- 29 総括

付録

- Appendix I 記号表
- Appendix II 定理依存関係
- Appendix III 原論文図表との対応
- Appendix IV 用語集
- 主要参考文献

本書について

対象論文と版

本書は、Scott Ames、Carmit Hazay、Yuval Ishai、Muthuramakrishnan Venkitasubramaniam による “**Ligero: Lightweight Sublinear Arguments Without a Trusted Setup**” の 2022 年 extended version を対象とする。これは CCS 2017 で公表された同名論文の完全版であり、構成の形式的証明、Reed–Solomon 符号テストの改善された soundness 解析、Fiat–Shamir 変換の解析、実装の改訂、後続研究との比較、未解決問題を含む。

原論文は、NP 関係に対し、検証回路サイズを s としたとき通信量がおおむね $\tilde{O}(\sqrt{s})$ となる、透明な public-coin zero-knowledge argument of knowledge を設計・実装する。暗号学的な外部仮定は、対話版では衝突困難ハッシュ、非対話版ではランダムオラクルとしてモデル化されたハッシュにほぼ限定される。中核は、拡張 witness を約 $\sqrt{s} \times \sqrt{s}$ の行列に配置し、各行を Reed–Solomon 符号化した **interleaved Reed–Solomon code** に対して、ランダム線形結合と少数列の開示だけで整合性を検査することである。

翻訳・解説方針

本書は、逐語置換型の翻訳ではなく、**定義、プロトコル、補題、定理、証明の論理、パラメータ依存性を落とさない注釈付き網羅訳**である。原論文の節構造を保ちつつ、次の処理を行った。

1. PDF 抽出時に崩れやすい添字、Hadamard 積、符号距離、二項係数、べき乗を標準的な数式表記に修復した。
2. 原文で暗黙に使われる符号理論、MPC-in-the-head、IPCP/IOP、Merkle/Fiat–Shamir の接続を補った。
3. soundness を「符号語から遠い」「近いが制約が偽」「誤り列を引かなかった」という事象に分解し、確率項の意味を説明した。
4. 先に扱った Aurora および 2018 年 ZK-STARK 論文と、共通の RS 符号・IOP・Merkle 基盤と相違点を明示した。
5. 後続の Brakedown、Shockwave、Spartan、Nova/HyperNova、Binius、STIR/WHIR、格子ベース sublinear argument まで含め、2026 年時点での技術的位置づけを追加した。ただし、異なる実装・ハードウェア・安全性水準のベンチマーク値を直接優劣として扱わない。

読者に仮定する前提

ZKP の完全性・健全性・零知識性の入門的説明は省略する。一方、本論文を読み解くために必要な以下の事項は、数式レベルで再導入する。

- ・ 有限体、補間多項式、Reed–Solomon 符号、最小距離
- ・ interleaved code と列距離
- ・ public-coin IPCP / IOP と oracle query
- ・ MPC の privacy、robustness、view consistency
- ・ Merkle commitment、Fiat–Shamir、ランダムオラクルモデル
- ・ 算術回路の拡張 witness と線形・二次制約への分解

事実・評価・推測の区別

本文中では、**原論文の主張**、**数学的帰結**、**本書の評価**を区別する。特に「post-quantum」は、公開鍵群演算を使わずハッシュ中心であることからの *plausible post-quantum security* を意味し、古典 ROM の証明が自動的に量子ランダムオラクルモデルへ移ることを意味しない。また、証明長が sublinear であることと、検証時間が polylogarithmic であることは別概念である。Ligero の検証器は、明示的に与えられた一般回路を処理するため、通常は回路サイズに線形の前処理を要する。

エグゼクティブ・サマリー

一文で表す Ligero

Ligero は、witness と中間 wire 値を平方根サイズの行列に詰め、行方向を Reed–Solomon 符号化し、ランダム線形結合と少数の列開示によって、符号所属・線形配線制約・乗算制約を同時に検査する、透明・ハッシュベースの平方根通信量 ZK argument である。

Ligero 全体構成

情報理論的 ZKIPCP を、コミットメントと公開乱数で実用的な argument に変換する

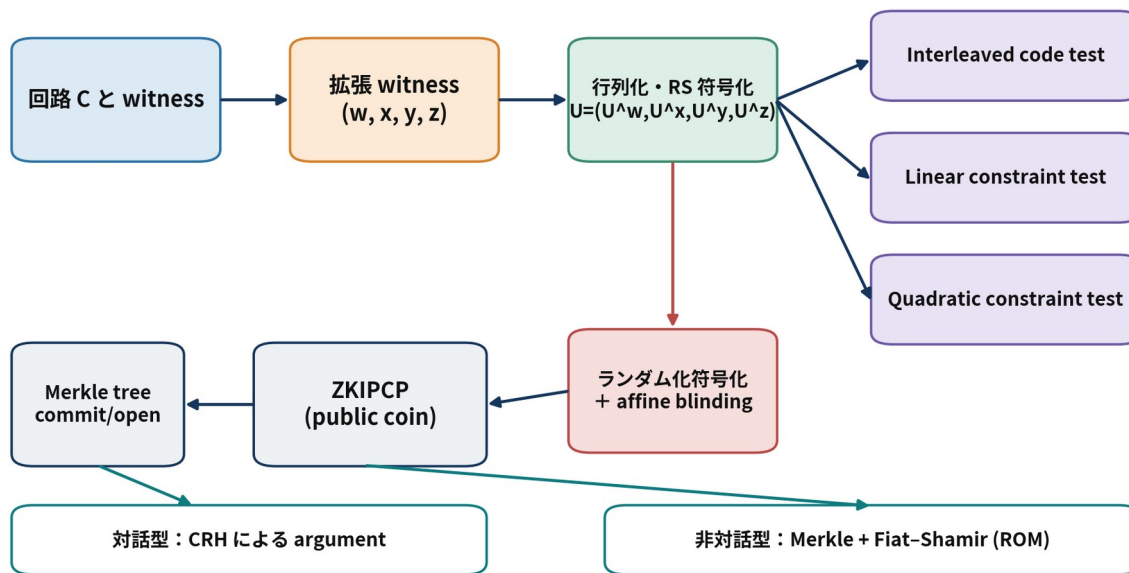


図 1 Ligero の情報理論的 ZKIPCP から、Merkle commitment と Fiat-Shamir を経て実用 argument へ至る全体構成。

主要特性

軸	Ligero の性質	読み方
対象	一般の Boolean/算術回路、したがって NP	R1CS 専用ではないが、回路を線形・二次制約へ変換する
setup	透明。対話版は CRH、非対話版は ROM	structured reference string を要求しない
通信量	$\tilde{O}(\sqrt{s})$	多項対数ではないが、大規模回路で回路本体より小くなる
prover	主に RS 符号化、補間、評価	算術回路では概ね $O(s \log s)$ 、構造に依存しにくい
verifier	oracle query 自体は sublinear、回路処理は一般に線形	明示回路を読む下限 $\Omega(s)$ がある
ZK	ZKIPCP では honest-verifier perfect ZK。コンパイル後は計算量的 ZK	ランダム化符号化と affine blinding が核心
安全性	情報理論的 IPCP soundness + ハッシュの binding	非対話化では ROM/QROM の区別が重要
強い用途	中規模から大規模、同一回路の batch、対称鍵中心、PQ 志向	証明を極小化したいオンチェーン用途には不利

技術的な三本柱

第一は **interleaved code test** である。 m 本の RS 符号語を $m \times n$ 行列 U と見なし、検証者はランダムな係数ベクトル $r \in F^m$ を送り、証明者から $w = r^T U$ を受け取る。 w が RS 符号語であることと、ランダムに選んだ t 列で $w_j = \sum_i r_i U_{i,j}$ が成り立つことを確認する。多数行の符号所属を一つのランダム線形結合へ圧縮する。

第二は **linear constraint test** と **quadratic constraint test** である。拡張 witness の複製・配線・加算関係を $Ax=b$ に、乗算ゲートを $x \circ y + a \circ z = b$ にまとめる。RS 符号語に対応する低次数多項式を用いると、制約全体を一つのランダム線形結合と一つの低次数多項式恒等式へ圧縮できる。

第三は **零知識化** である。開かれる少数列を witness から独立にするため、同じ message に復号される高次数のランダム符号語を選ぶ。さらに、検証者へ送る線形結合多項式をランダム符号語で affine に blind する。これにより、シミュレータは開示位置の値と応答多項式を witness なしで直接サンプリングできる。

Ligero を理解するための最重要ポイント

1. 平方根は偶然ではない。 $m\ell \simeq s$ の行列化で、行方向の多項式応答コストが $O(\ell)$ 、列開示コストが $O(tm) = O(ts/\ell)$ となるため、 $\ell \simeq \sqrt{ts}$ で最小化される。
2. RS 符号は **commitment** ではない。情報理論的層では oracle として扱われ、実用層で列を Merkle 木へ commit する。
3. **soundness** は三種類のランダム性から生じる。行のランダム線形結合、制約のランダム線形結合、ランダム列 query である。
4. ZK のための次数余裕が通信量へ跳ね返る。条件 $k > \ell + t$ は、 t 個の評価値を一様に隠すための自由度を確保する。
5. Aurora/STARK との違いは低次数検査の使い方にある。Ligero は平方根個の列を直接開く一段型 IPCP、Aurora/STARK は FRI 系の多段 IOP で query 数を対数化する。

Part I 背景と数学的準備

1 Ligero が位置する証明系の系譜

1.1 2017 年前後の三つの流れ

原論文は、実用的な汎用計算証明を三群に分ける。

Doubly efficient interactive proofs

GKR 系は、層状回路に対する sumcheck を用い、通信量と検証計算を大幅に削減する。低深度・高並列な回路、同一回路の反復、streaming verification に強い。一方、一般 NP へ拡張する場合、回路深度依存や polynomial commitment のコストが現れやすい。

PCP / IPCP / IOP

PCP では検証者が巨大な証明を全読せず、少数位置のみ照会する。IPCP は最初に PCP oracle を送り、その後に短い対話を行う。IOP は各 round で oracle message を送れる一般化である。Kilian/Micali 型コンパイラにより、oracle を Merkle 木へ commit して実用 argument にできる。透明性とハッシュ中心の安全性が長所だが、古典 PCP は具体的定数が大きかった。

Linear PCP + homomorphic public-key cryptography

QAP/QSP 系 SNARK は、線形 PCP を pairing や homomorphic encoding で暗号化し、定数個の群要素へ圧縮する。Groth16 に代表される方式は極めて短い証明と高速検証を実現するが、回路依存の structured setup や量子耐性の欠如が代償となる。

Ligero の目標は、PCP/IOP 系の透明性・対称鍵性を保ちつつ、MPC 由来の単純な局所整合性検査で具体効率を改善することである。

1.2 MPC-in-the-head との関係

MPC-in-the-head では、証明者が複数 party の MPC 実行をローカルに模擬し、その view へ commit する。検証者はランダムに一部 party の view を開かせ、互いに整合していることを確認する。ZKBoo のような典型方式では、party 数を小さく固定して反復により soundness を下げる。

Ligero の発想はこれを「多数 party」と「符号化された view」に拡張する。MPC の全通信量が回路サイズ s 程度で、party 数を n_p とすると一 party 当たり view は約 s/n_p となる。開く party 数を security parameter 相当にすると、通信量は概ね

$$Comm(n_p) \approx n_p + \kappa \frac{s}{n_p},$$

となり、 $n_p \approx \sqrt{\kappa s}$ で平方根になる。直接構成では party view を interleaved RS 行列として表現し、この直観を代数的な検査へ落とし込む。

MPC から ZKIPCP への変換

「多数のサーバのうち少数だけを開く」watchlist / MPC-in-the-head の見方

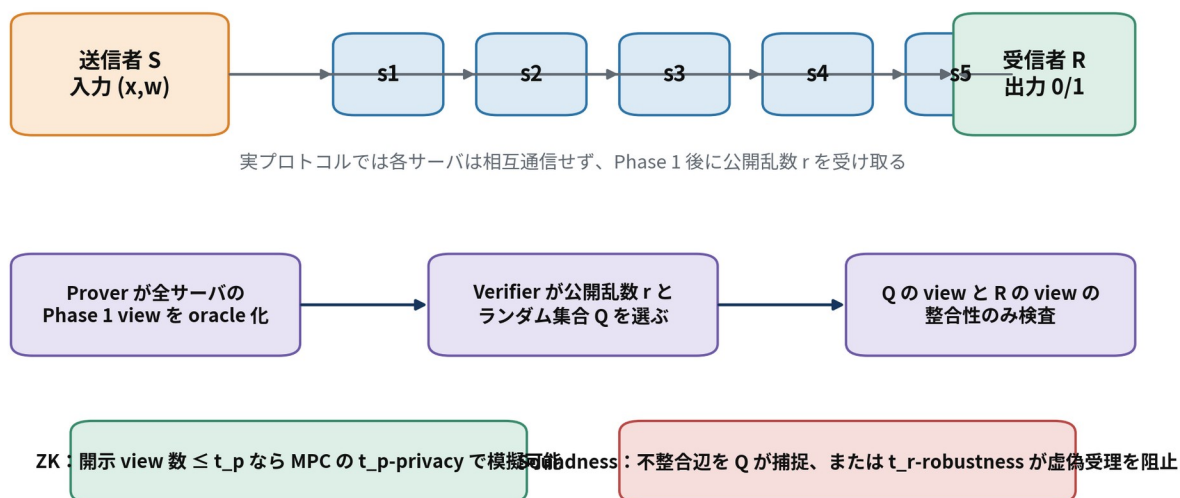


図2 送信者・サーバ・受信者からなる制限 MPC を head 内で実行し、ランダムに選んだサーバ view を開く一般変換。

1.3 「succinct」の語に関する注意

原論文は zk-SNARK という語を用いるが、今日の実装文脈では「証明長と検証時間の双方が polylogarithmic」を SNARK の典型像と考えることが多い。Ligero の証明長は $\tilde{O}(\sqrt{s})$ 、検証時間は一般に $O(s)$ である。したがって、本書では次を使い分ける。

- **sublinear argument**：証明・通信が witness または回路サイズより漸近的に小さい。
- **polylog succinct argument**：証明長が polylogarithmic。
- **doubly succinct/scalable verifier**：検証時間も polylogarithmic。

Ligero の価値は「極小証明」ではなく、**setup なし、対称鍵中心、単純、回路構造に依存しにくい、平方根通信** という設計点にある。

2 符号理論と多項式表現

2.1 Reed-Solomon 符号

有限体 F 、相異なる評価点

$$\eta = (\eta_1, \dots, \eta_n) \in F^n$$

を固定する。 $[n, k, d]$ Reed-Solomon 符号は

$$RS_{F, n, k, \eta} = \{ (p(\eta_1), \dots, p(\eta_n)) : \deg p < k \}$$

であり、最小距離は $d = n - k + 1$ である。異なる二つの次数 $< k$ の多項式は高々 $k - 1$ 点でしか一致しないため、対応する符号語は少なくとも $n - k + 1$ 座標で異なる。

Ligero は、message 座標と codeword 座標を分ける。message 評価点

$$\zeta = (\zeta_1, \dots, \zeta_\ell)$$

を選び、符号語 $u = (p(\eta_1), \dots, p(\eta_n))$ の復号 message を

$$Decode_\zeta(u) = (p(\zeta_1), \dots, p(\zeta_\ell))$$

と定める。 $\ell < k$ なら、同じ message に復号される符号語は多数存在する。この非一意性が零知識化に使われる。

2.2 interleaved code

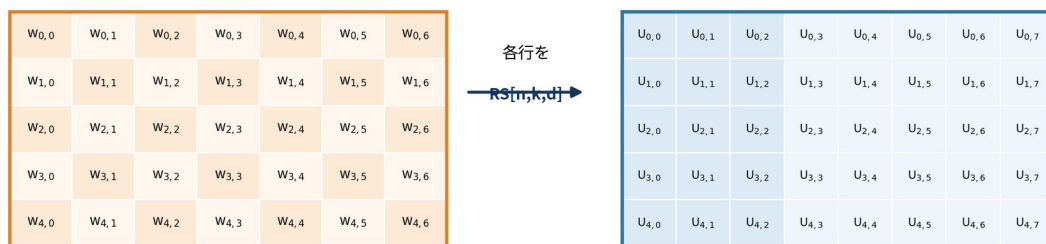
m 本の符号語を行に並べた行列

$$U = \begin{bmatrix} U_1 \\ \vdots \\ U_m \end{bmatrix} \in F^{m \times n}, U_i \in L$$

を L^m の符号語とみなす。alphabet は列ベクトル F^m であり、Hamming 距離は列単位で数える。すなわち、一列に一つでも誤りがあればその列全体を誤りと数える。この定義は、検証者が列をまとめて Merkle open する実装と一致する。

拡張 witness の行列化と interleaved RS 符号

$\sqrt{s}$ の通信量が現れる幾何学的な理由



m を小さくすると列開示が安いが、 ℓ と k が増え応答多項式が大きくなる。
両者を釣り合わせると $m, \ell \approx \sqrt{|C|}$ 。

図3 $w \in F^{m\ell}$ を $m \times \ell$ に配置し、各行を長さ n の RS 符号語へ延長する。

2.3 距離・近傍・一意復号半径

ベクトル $v \in F^n$ と符号 L の距離を

$$d(v, L) = \min_{c \in L} |\{j : v_j \neq c_j\}|$$

とする。 $e < d/2$ なら、 e 以内の符号語は一意である。Ligero の soundness 解析は、「oracle 行列が L^m から e より遠い場合」と「 e 以内に近い場合」に分ける。

この分割は本質的である。遠い場合は interleaved test で弾く。近い場合は、最寄りの正しい符号語行列が復号する message に対して、線形・二次制約が偽であることを検出する。

2.4 ランダム線形結合の圧縮原理

U の行に不正があるとする。ランダム $r \in F^m$ に対する

$$w = r^\top U = \sum_{i=1}^m r_i U_i$$

が偶然「正しい符号語に近い」確率は小さい。直観的には、ある不正方向 v^i に沿う affine line $x + \alpha v^i$ 上で、符号の近傍球に入る α は少数しかない。これが Lemma 4.2、4.5、4.6 の幾何学的内容である。

3 IPCP、IOP、Merkle、Fiat-Shamir

3.1 IPCP の構文

IPCP では、証明者が最初に長い oracle π を固定し、その後、検証者と短い public-coin 対話を行う。最後に検証者は π の少数位置を照会する。効率指標は次である。

- oracle length：情報理論的証明全体の長さ
- query complexity：読まれる oracle symbol 数
- interaction communication：oracle 以外の対話量
- round complexity
- prover/verifier time

Ligero の ZKIPCP は、oracle 長自体は線形だが、実用 argument では Merkle root だけを送って必要列のみ open するため、実通信量が平方根になる。

3.2 Merkle commitment による oracle の実現

oracle の各列を leaf として Merkle 木へ commit する。検証者が列 j を照会すると、証明者は列値と authentication path を返す。衝突困難性により、一度送った root に対して相矛盾する値を開くことは困難である。

列単位で commit する理由は、interleaved code の alphabet が列 $U[j] \in F^m$ だからである。一つの path で、その列に含まれる witness、線形像、blinding row をまとめて認証できる。

3.3 Fiat-Shamir と BCS 型変換

public-coin IOP/IPCP の各 verifier message を、そこまでの transcript のハッシュ

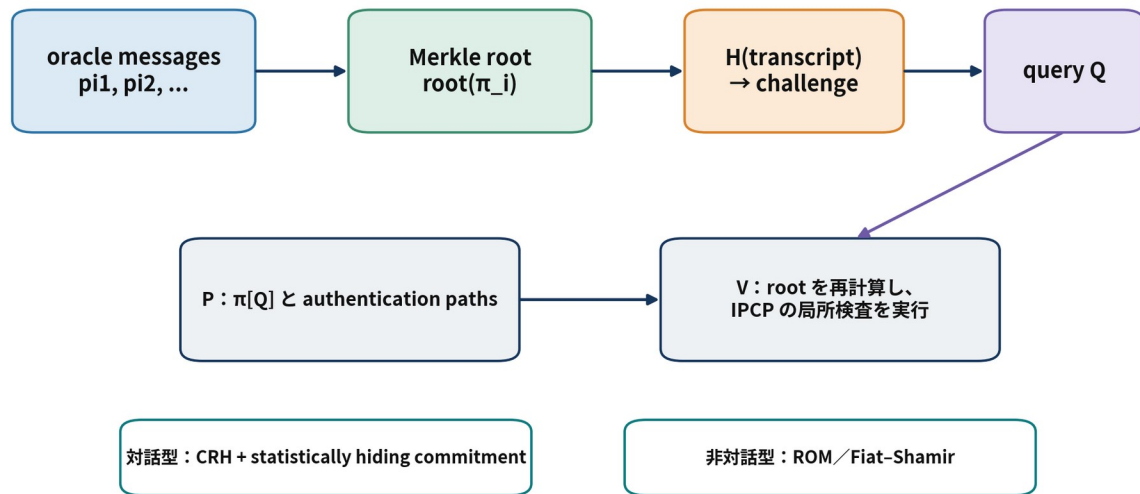
$$r_i = H(\text{domainsep} \parallel \text{transcript}_{i-1})$$

から導出する。oracle message は Merkle root で置き換え、query 位置もハッシュから導出する。これにより一つの proof message になる。

ただし、単純な「challenge をハッシュで置換すればよい」という説明だけでは不十分である。原論文は BCS16 の解析を用い、state-restoration soundness と random-oracle query collision を考慮する。古典 ROM では、悪意ある prover の RO query 数を q_H 、ハッシュ出力長を κ とすると、変換に伴う追加損失は概ね $O(q_H^2 2^{-\kappa})$ 型である。

ZKIPCP から実用 argument へ

oracle access を Merkle 開示に、verifier の公開乱数を Fiat-Shamir に置き換える



注意：古典 ROM の証明と量子ランダムオラクル模型の証明は同一ではない

図4 oracle を Merkle root へ圧縮し、transcript hash で public coin と query を生成する。

3.4 量子安全性についての厳密な言い方

Ligero の代数層は情報理論的であり、実用層はハッシュに依存する。したがって、離散対数・pairing に基づく方式よりも量子計算に対して有望である。しかし次を区別する必要がある。

1. ハッシュの量子攻撃コスト：Grover 探索により preimage security は指数が半減し、collision search にも量子加速がある。出力長を安全性目標に合わせて設定する必要がある。
2. Fiat-Shamir の証明モデル：古典 ROM での変換定理と QROM での証明は同一ではない。
3. 実装 hash の構造：理想 random oracle と具体 hash の間には仮定がある。

したがって、本書では「ハッシュベースで plausibly post-quantum」と表現する。

Part II 原論文の注釈付き網羅訳

4 Abstract・Introduction・Our Results

4.1 Abstract の注釈付き翻訳

本論文は、NP に対する単純な zero-knowledge argument protocol を設計・実装する。その通信量は、witness を検証する回路サイズの平方根に比例する。対話版は任意の衝突困難ハッシュ関数に基づける。また、ランダムオラクルモデルで非対話化でき、trusted setup も public-key cryptography も不要な、具体的に効率のよい zk-SNARK を得る。

プロトコルは、Ishai らの MPC-to-ZK 変換を最適化し、Damgård–Ishai の MPC protocol の変種へ適用して得られる。別の見方をすれば、これは **interleaved Reed–Solomon code** に基づく単純な zero-knowledge interactive PCP である。

2022 年版は CCS 2017 版の拡張であり、より鋭い soundness 解析、改善された実装、完全な形式的証明を含む。検証回路が大きい場合、Ligero prover は後続方式と比較しても競争力があり、同一回路で複数 instance を証明する amortized setting では長所が増す。

具体例として、soundness error 2^{-40} で SHA-256 preimage を証明する通信量は約 35 KB である。通信量は回路トポロジーではなく gate 数に主として依存し、同じ安全性では約 300 万 gate 以上で回路サイズを下回る。改善された解析により、小規模から中規模の回路では、後続の post-quantum ZK-SNARK より proof length と prover time が良好となる領域がある。

注釈

- ・ 「任意の CRH」は black-box 利用を指し、hash 内部を算術回路として証明する必要がある。
- ・ 「zk-SNARK」は当時の用語法であり、proof が sublinear という意味が強い。Ligero の proof は polylogarithmic ではない。
- ・ 35 KB は 2017/2022 論文の特定 parameter と実装に基づく歴史的値であり、128-bit 安全性や別 field では増える。

4.2 Introduction の翻訳と背景

外部委託計算では、計算者が不正な結果を報告する誘因を持ち得る。Ligero は、この計算完全性を、検証回路サイズの平方根に比例する通信量で保証する。得られるのは単なる argument ではなく、zero-knowledge argument of knowledge であり、verifier message は public coin のみである。したがって Fiat–Shamir で非対話化できる。

論文は既存研究を三群に整理する。

1. **doubly efficient IP**：GKR に始まり、低深度計算に対して小通信・高速検証を達成する。
2. **PCP/IPCP/IOP**：公開鍵暗号や trusted setup を避け、post-quantum 候補となるが、当時は具体効率が課題だった。
3. **linear PCP + homomorphic PKC**：短い proof を実現する一方、prover の群演算負担と structured CRS を要する。

Ligero の目標は、setup、複雑な PCP machinery、高価な公開鍵演算なしに、単純・具体効率的・sublinear communication の public-coin ZK argument を得ることである。

4.3 Our Results の翻訳

主結果は次の特徴を同時に満たす。

- ・ 通信量は検証回路サイズのほぼ平方根。
- ・ 自己完結的に記述・解析できるほど単純。

- ・ 対称鍵 primitive、具体的には CRH を black-box 利用するだけ。
- ・ public coin であり、ROM では Fiat-Shamir により公開検証可能な非対話 proof となる。
- ・ 非対話版を含め trusted setup 不要。
- ・ 実装により具体効率を示す。
- ・ 同一回路の複数 instance では、amortized communication と verification を改善できる。

構成上の出発点は IKOS 変換である。ただし、任意の honest-majority MPC をそのまま使うのではなく、送信者 S 、多数の server $s_1, \dots, s_n$ 、受信者 R からなる制限 network を使う。server 間通信を除き、最初の input 配布と最後の output 送信だけにする。Damgård–Ishai 系 MPC の総通信量は party 数にほぼ依存せず回路サイズに比例するため、party 数を平方根に設定すると、一 party の view も平方根サイズになる。

Informal Theorem 1.1

衝突困難ハッシュが存在すると仮定する。すると回路 C の充足可能性を証明する public-coin zero-knowledge argument が存在し、その通信量は

$$\tilde{O}(\sqrt{|C|})$$

である。

具体効率の主張

算術回路に対する通信量は、field size、安全性 parameter、回路サイズを含む式で与えられ、大 field では概ね

$$O(|F| \sqrt{|C|} \kappa) \text{ bits}$$

型、小 field では反復回数が増える。30-bit prime 上では、40-bit soundness で約 260 万 gate、128-bit で約 2000 万 gate を超えると、通信量が回路自体より小さくなると報告される。

同一回路の N instance では、 $N = \Omega(\kappa^2)$ 程度から一 instance 当たり通信量が回路サイズを下回り、総検証時間は

$$O(s \log s + N \log N)$$

field operations へ amortize できる。

4.4 後続研究に関する 2022 年版の追記

2022 年版は Aurora、GKR 系 transparent argument、Ligero と Aurora の hybrid、MPC-in-the-head signature、Brakedown/Shockwave 系 linear-time IOP などを概観する。ここで重要なのは、Ligero の設計点が後続方式に置き換えられたのではなく、**proof size**、**prover overhead**、**verifier time**、**field flexibility** の **trade-off 空間の一端**として残っていることである。

5 Preliminaries の注釈付き翻訳

5.1 記法

security parameter を κ とする。NP 関係 R に対して

$$R_x = \{w : (x, w) \in R\}, L_R = \{x : \exists w (x, w) \in R\}.$$

関数は Boolean 回路または算術回路で表す。回路サイズ $|C|$ は、NOT を除く gate と input gate の総数とする。

5.2 Collision-resistant hashing と Merkle tree

深さ d の二分 Merkle 木は $\ell = 2^d$ 個の message を一つの root h へ commit する。leaf m_i を開くには、root までの path 上の sibling hash を示せばよく、開示量は $O(d) = O(\log \ell)$ である。

binding は hash の collision resistance へ帰着する。ROM では hash を random oracle に置き換え、collision 発見の困難性から statistical binding を議論する。

Ligero での役割

- leaf は単一 field element ではなく、同一列に属する複数 field element をまとめた symbol にできる。
- proof size の一部は、開く列数 t と tree depth $\log n$ の積である。
- 同じ列を三つの subtest で共有するため、認証 path を共有できる。

5.3 Interactive argument と zero knowledge

原論文は、PPT prover/verifier に対する計算量的 soundness と、auxiliary input を許す computational ZK を定義する。直接構成の ZKIPCP は honest-verifier perfect ZK であり、その後の commitment/Fiat-Shamir 変換で一般の argument へ移す。

5.4 IOP と IPCP

IOP の各 round では、verifier が一様な public message を送り、prover が oracle message π_i を返す。全 round 後に verifier が各 oracle の少数位置を query する。

IPCP は、最初の round にだけ長い proof oracle を送る特殊な IOP である。その後の prover message は短い通常 message でよい。Ligero はこの IPCP 構造を使うため、oracle 全体は一度だけ commit される。

5.5 Zero-knowledge IPCP

honest verifier の view を、witness なしで完全に同一分布として生成する expected polynomial-time simulator が存在することを要求する。Ligero の simulator は rewinding を使わず、verifier randomness、query set、開示値、応答多項式を直接組み立てる。

6 From MPC to ZKIPCP

6.1 制限 MPC model

参加者は次の通りである。

- sender client S ：public statement x と witness w を持つ。
- n servers $S_1, \dots, S_n$ ：sender から share/view を受け、local computation を行う。
- receiver client R ：最終 output を受け取る。

network は意図的に制限される。

1. Phase 1 の冒頭で S から各 server へ一回だけ message を送る。
2. server 間通信はない。
3. Phase 1 後、全 server へ public random string r が broadcast される。
4. Phase 2 の最後に各 server が R へ一回だけ message を送る。

この形にすると、各 server view がほぼ独立な oracle symbol となり、ランダムに一部を開く MPC-in-the-head 検査へ直接変換できる。

6.2 必要な安全性概念

t_p -privacy

受信者と高々 t_p server が passive に corrupt されても、その joint view は入力・出力から simulator が完全に再現できる。ZK はこの privacy へ帰着する。

statistical t_r -robustness

sender と高々 t_r server が active かつ adaptively corrupt されても、真の witness が存在しないとき receiver が 1 を出す確率は negligible である。soundness はこの robustness へ帰着する。

consistent views

二つの view 間で、片方の outgoing message と他方の incoming message が一致すること。view consistency graph で不一致 edge を数える。

6.3 一般変換プロトコル Π_{ZKIPCP}

Oracle 生成

prover は MPC を head 内で実行し、Phase 1 終了時点の各 server view

$$(V_1, \dots, V_n)$$

を oracle の n symbols として固定する。

対話

1. verifier は public random challenge r を送る。
2. prover は receiver R の view を送る。
3. verifier は receiver output が abort せず 1 であることを確認し、 $[n]$ から t_p 個の server index をランダムに選ぶ。

4. oracle から選択 server の view を読む。
5. receiver view と各 server view が不整合なら reject し、そうでなければ accept する。

Theorem 3.5

基礎 MPC が statistical t_r -robustness と perfect t_p -privacy を持つとき、上記は ZKIPCP となり、soundness error は

$$\left(1 - \frac{t_r}{n}\right)^{t_p} + \delta(\kappa)$$

以下である。 δ は MPC robustness error である。

6.4 soundness proof : inconsistency graph

証明の要点は server と receiver を vertex とする graph G である。不整合な view pair に edge を張る。

Case 1：不整合 edge が t_r より多い

receiver view は常に検証者へ見えているため、不整合に関与する server を一つでも開けば reject する。 t_p 回の抽出がすべて不整合 server を外す確率は高々

$$\left(1 - \frac{t_r}{n}\right)^{t_p}.$$

Case 2：不整合 edge が t_r 以下

cheating prover を用いて、MPC に対する active adversary を構成する。adversary は sender を corrupt し、prover が oracle に固定した server input を外部 MPC へ注入する。receiver view と食い違う server だけを最後に adaptively corrupt し、その message を置換する。不整合 server 数は t_r 以下なので、MPC robustness により false statement で receiver が 1 を出す確率は $\delta(\kappa)$ 以下である。

この二 case を union bound して定理を得る。

6.5 zero knowledge proof

verifier が見るのは receiver view と t_p server view だけである。基礎 MPC の t_p -privacy simulator に、receiver および選択 server を corrupt させれば、witness なしで同一分布の view を生成できる。

6.6 通信量の最適化

一 server view の最大長を v_{size} 、receiver view を v_R とすると prover communication は

$$t_p v_{size} + v_R.$$

基礎 MPC の設計により

$$v_{size} v_R = O(|C|)$$

となるよう parameter を選べる。二項を同程度にすると平方根が得られる。直接 ZKIPCP 構成は、この抽象的な MPC view を RS 符号語行列として具体化したものである。

7 A Direct ZKIPCP Construction

7.1 高水準の構成

回路の input と各 gate output を並べた extended witness を

$$w \in F^{m\ell}, m\ell > n_i + s$$

として、 $m \times \ell$ 行列へ配置する。各行を message とし、RS 符号 $L = RS_{F,n,k,\eta}$ で長さ n へ符号化する。得られる $m \times n$ 行列が oracle である。

検証は bottom-up に三層へ分かれる。

1. 各行が同じ RS 符号に属するか。
2. 復号 message が配線・複製・加算という線形制約を満たすか。
3. 復号 message が乗算という二次制約を満たすか。

三 test は同じ t 列を開き、authentication cost を共有する。

7.2 Test-Interleaved

Protocol

oracle を $U \in F^{m \times n}$ とする。

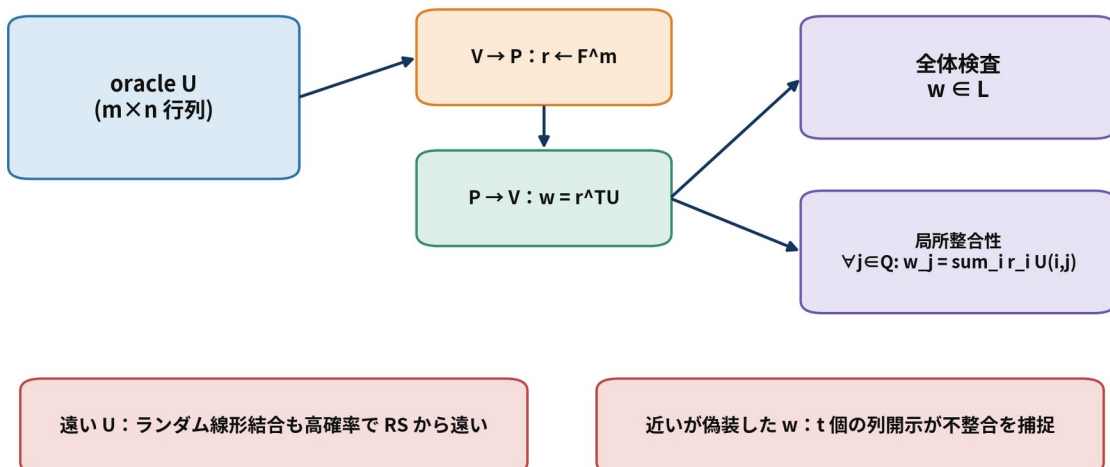
1. V は $r \leftarrow F^m$ を送る。
2. P は $w = r^T U \in F^n$ を返す。
3. V は一様な query set $Q \subset [n], |Q|=t$ を選び、 $U[j]$ を読む。
4. $w \in L$ かつ全 $j \in Q$ で

$$w_j = \sum_{i=1}^m r_i U_{i,j}$$

なら accept する。

Test-Interleaved

多数の行の「すべて RS codeword」を 1 本のランダム線形結合へ圧縮する



典型的な誤り上界： $(1-e/n)^t + \text{code-combination term}$

図5 多数行の RS 所属を一つのランダム線形結合へ圧縮し、少数列で oracle との整合性を確認する。

Completeness

$U \in L^m$ なら線形性により $r^\top U \in L$ であり、全列で等式が成り立つ。

Lemma 4.2 の内容

$e < d/4$ かつ $d(U^i, L^m) > e$ とする。 U^i の row span から一様選んだ w^i が L から e 以内となる確率は

$$\Pr[d(w^i, L) \leq e] \leq \frac{e+1}{|F|}.$$

証明は二 case に分かれる。

1. row span に L から $2e$ より遠い方向 v^i がある。 $w^i = x + \alpha v^i$ と書くと、 e -近傍へ入る α は高々一つ。二つあれば差 $(\alpha - \alpha')v^i$ が $2e$ -近傍に入り矛盾する。
2. row span の全ベクトルが $2e$ 以内。各行の誤り support を合併した集合 E は e より大きい。 E の先頭 $e+1$ 座標について、誤りが相殺される係数は各座標高々一つなので union bound する。

Theorem 4.4

遠い oracle が accept される確率は

$$\left(1 - \frac{e}{n}\right)^t + \frac{e+1}{|F|}$$

以下である。第一項は e 個以上ある不整合列をすべて外す確率、第二項はランダム線形結合が偶然近い符号語に落ちる確率である。

7.3 改善解析： $e < d/3$ 、さらに $e < d/2$

CCS 2017 版は $e < d/4$ を用いた。2022 年版では Roth-Zémor の affine-line 補題を取り込み、RS 符号について $e < d/3$ へ改善する。

Lemma 4.4 (affine line)

RS 符号 L と affine line

$$\ell_{u,v} = \{u + \alpha v : \alpha \in F\}$$

を考える。 $e < d/3$ なら、

- ・ line 上の全点が L から e 以内、または
- ・ e 以内の点は高々 d 個

のどちらかである。

これにより

$$\Pr[d(w^i, L) \leq e] \leq \frac{d}{|F|}$$

となり、interleaved test の error は

$$\left(1 - \frac{e}{n}\right)^t + \frac{d}{|F|}$$

へ改善される。

後続の BCI+20 解析では一意復号半径 $e < d/2$ まで拡張し、field 項を $n/|F|$ とする。field が十分大きければ、より高 rate の RS 符号を使えるため proof が短くなる。

7.4 Linear constraint test

7.4.1 問題設定

$U \in L^m$ が message $x \in F^{m\ell}$ を符号化しているとする。既知の sparse matrix A と vector b に対して

$$Ax=b$$

を検査したい。全制約を直接読む代わりに、ランダム $r \in F^{m\ell}$ で

$$r^T Ax = r^T b$$

を検査する。 $Ax \neq b$ なら、これは確率 $1/|F|$ でしか成立しない。

7.4.2 多項式化

$r^T A$ を m block、各 block ℓ 個へ分ける。block i の係数を、 $\zeta_1, \dots, \zeta_\ell$ 上で補間する次数 ℓ 多項式 $r_i(X)$ とする。 U_i に対応する次数 k 多項式を $p_i(X)$ とすると、

$$q(X) = \sum_{i=1}^m r_i(X) p_i(X)$$

は次数 $k+\ell-1$ である。

ζ_c で評価すると

$$\sum_{c=1}^{\ell} q(\zeta_c) = (r^T A)x.$$

したがって、prover は q の係数を送り、verifier は

$$\sum_c q(\zeta_c) = r^T b$$

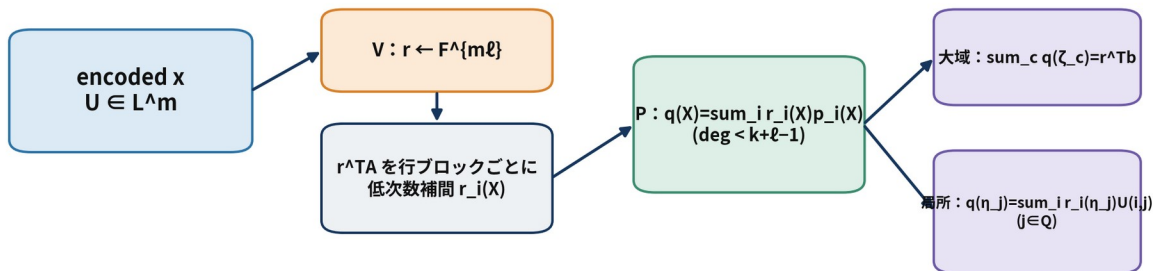
と、random query 列で

$$q(\eta_j) = \sum_i r_i(\eta_j) U_{i,j}$$

を確認する。

線形制約テスト $Ax=b$

巨大な制約系をランダム線形結合 1 本と、低次数多項式 q に落とす



$Ax \neq b$ なら $r^T(Ax-b)=0$ となる確率は $1/|F|$ 。
偽 q は次数境界により多数点で真の q と一致できず、ランダム列で露見する。

図6 多数の線形制約をランダム線形結合し、低次数多項式 q の global sum と局所列整合性へ変換する。

7.4.3 soundness

U^i が正しい codeword U から高々 e 列ずれ、 U が復号する x は $Ax \neq b$ とする。悪意ある prover が accept される確率は

$$\frac{1}{|F|} + \left(\frac{e+k+\ell}{n} \right)^t$$

以下である。

- $1/|F|$ ：偽の線形関係がランダム圧縮で消える確率。
- 第二項：偽の q' と正しい q が一致し得る高々 $k+\ell-2$ 点と、oracle の e 誤り列の union から、全 query を選んでしまう確率。

Appendix C の joint analysis では、interleaved test が誤り列を引いていないと条件付けることで e を除き、

$$\frac{1}{|F|} + \left(\frac{k+\ell}{n} \right)^t$$

へ改善する。

7.5 Quadratic constraint test

7.5.1 問題設定

三 message $x, y, z \in F^{m\ell}$ が

$$x \circ y + a \circ z = b$$

を満たすか検査する。 a, b は public で、 $\circ$ は成分ごとの積である。

各 row i の多項式を p_i^x, p_i^y, p_i^z 、public vector の符号多項式を p_i^a, p_i^b とすると

$$p_i(X) = p_i^x(X) p_i^y(X) + p_i^a(X) p_i^z(X) - p_i^b(X)$$

は次数 $\leq 2k-1$ である。ランダム $r \in F^m$ により

$$p_0(X) = \sum_{i=1}^m r_i p_i(X)$$

を作る。制約が真なら、全 message point ζ_c で $p_0(\zeta_c) = 0$ である。

二次制約テスト $x \odot y + a \odot z = b$

乗算ゲート群を RS 多項式の積とランダム線形結合で一括検査する

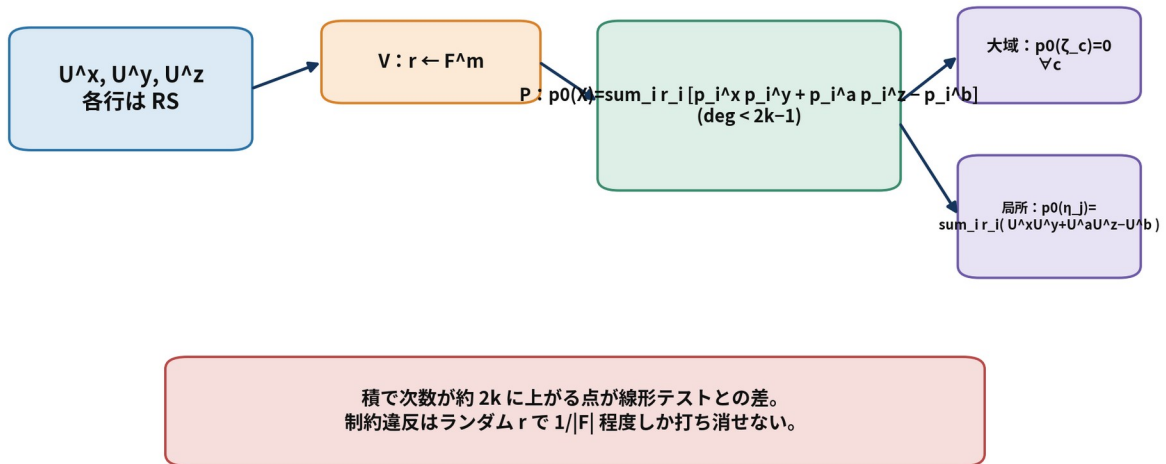


図7 Hadamard 積制約を次数 $< 2k - 1$ の一つのランダム多項式へ圧縮する。

7.5.2 protocol

1. verifier が $r \leftarrow F^m$ を送る。
2. prover が p_0 の $2k - 1$ 個の係数を送る。
3. verifier が同じ query set Q の列を開く。
4. 全 $c \in [\ell]$ で $p_0(\zeta_c) = 0$ 、全 $j \in Q$ で

$$p_0(\eta_j) = \sum_i r_i (U_{i,j}^x U_{i,j}^y + U_{i,j}^a U_{i,j}^z - U_{i,j}^b)$$

を確認する。

7.5.3 soundness

近い正しい codeword が復号する x, y, z が制約を破るとき、accept 確率は

$$\frac{1}{|F|} + \left(\frac{e+2k}{n} \right)^t$$

以下である。Appendix C の joint analysis では

$$\frac{1}{|F|} + \left(\frac{2k}{n} \right)^t$$

へ改善される。

7.6 算術回路への合成

回路 $C: F^{n_i} \rightarrow F$ の input α と全 gate output $\beta_1, \dots, \beta_s$ を extended witness w に入れる。

各 multiplication gate について入力値・出力値を集めた vector を x, y, z とし、selection matrix P_x, P_y, P_z で

$$x = P_x w, y = P_y w, z = P_z w$$

と書く。加算 gate は $P_{add} w = 0$ へまとめる。乗算は

$$x \circ y - z = 0$$

である。

prover は w, x, y, z の符号行列 U^w, U^x, U^y, U^z を縦に連結し、次を並列実行する。

1. Test-Interleaved：全行の符号所属。
2. Linear-Constraints： $x = P_x w, y = P_y w, z = P_z w, P_{add} w = 0$ 。
3. Quadratic-Constraints： $x \circ y - z = 0$ 。

Lemma 4.12 / Theorem 4.6

false circuit に対する basic IPCP の soundness error は、 $e < d/3$ の解析では

$$\frac{d+2}{|F|} + \left(1 - \frac{e}{n}\right)^t + 2 \left(\frac{e+2k}{n}\right)^t$$

で上から抑えられる。通信される field element 数は、interleaved 応答 k 、linear 応答 $k + \ell - 1$ 、quadratic 応答 $2k - 1$ の和である。oracle からは t 列を読む。

7.7 Boolean 回路への適用

Boolean bit β を field element として表し、

$$\beta^2 - \beta = 0$$

で $\beta \in \{0, 1\}$ を強制する。

XOR と AND は補助 bit d を使って線形化できる。

$$\bullet \quad b_3 = b_1 \oplus b_2 :$$

$$b_1 + b_2 = b_3 + 2d.$$

$$\bullet \quad b_3 = b_1 \wedge b_2 :$$

$$b_1 + b_2 = d + 2b_3.$$

すべての変数が bit である限り、これらは整数として正しい。十分大きい characteristic の prime field で検査すればよい。

32-bit 加算 $a + b = c \pmod{2^{32}}$ も carry bit d を用いて

$$\sum_{i=0}^{31} 2^i a_i + \sum_{i=0}^{31} 2^i b_i = 2^{32} d + \sum_{i=0}^{31} 2^i c_i$$

と線形化できる。ただし wraparound を避けるため characteristic $\geq 2^{33}$ が必要である。

7.8 Achieving Zero-Knowledge

basic IPCP は二種類の情報を漏らす。

1. $r^\top U$ のような codeword の線形結合。
2. query 列の局所値。

局所値は **randomized encoding** で隠す。message 評価点 ζ で固定値を取りつつ、次数 k の残り自由係数を一様を選ぶ。 $k > \ell + t$ なら、 L 上の任意の t 評価は一様かつ message から独立になる。

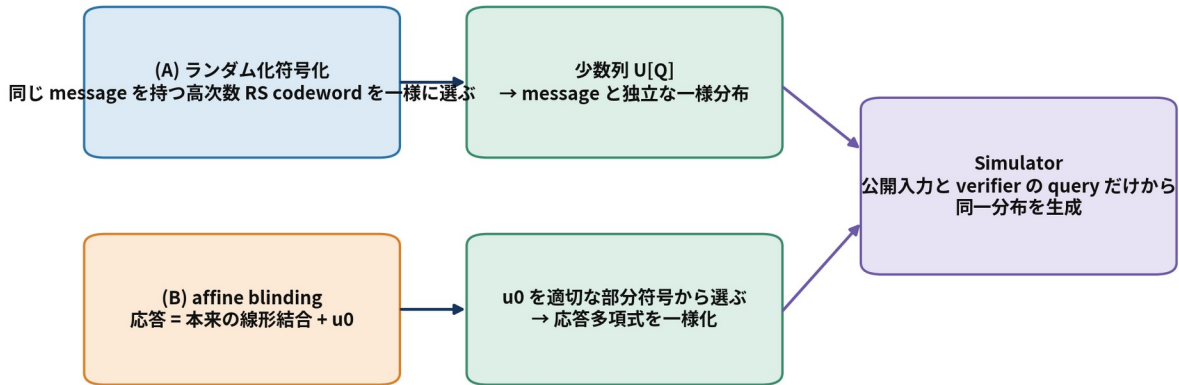
線形結合は **affine blinding** で隠す。例えば interleaved test の応答を

$$v = r^\top U + u^0$$

とし、 u^0 を適切な message を符号化するランダム codeword にする。係数をランダムに掛けるのではなく固定係数 1 で加えることで、blinding が消える事象を避ける。

完全 ZK を得る二つの遮蔽

局所開示には randomized encoding、全体線形結合には affine blinding を使う



鍵条件： $k > \ell + t$ （開示する t 列より十分な自由度を残す）

図8 randomized encoding が局所 query を隠し、affine blinding が global linear response を隠す。

Linear test の blind

追加 row u^{add} は、message $(\gamma_1, \dots, \gamma_\ell)$ が

$$\sum_{c=1}^{\ell} \gamma_c = 0$$

を満たすよう一様を選ぶ。応答多項式へ $r_{Blind}^{add}(X)$ を加えても global sum は変わらないが、個々の $q(\zeta_c)$ は隠れる。

Quadratic test の blind

追加 row u^0 は zero message を符号化し、その対応多項式を p_0 へ加える。message 点で 0 なので制約検査を変えず、codeword 上では応答をランダム化する。

7.9 Final ZKIPCP

soundness を小 field でも下げるため、 σ 個のランダム線形結合を並列に行う。各 $h \in [\sigma]$ について、interleaved、linear、quadratic の challenge と blind row を生成する。すべて同じ query set Q を使う。

Lemma 4.14

false circuit に対する accept 確率は basic 解析で

$$\frac{d+2}{|F|^\sigma} + \left(1 - \frac{e}{n}\right)^t + 2 \left(\frac{e+2k}{n}\right)^t$$

以下となる。 σ は field 由来の Schwartz-Zippel 型 error を指数的に下げ、 t は列 miss error を指数的に下げる。

Lemma 4.15 : perfect HVZK

条件 $k > \ell + t$ の下で、simulator は各反復について次を一様を選べる。

- $\sum_c q_h^{add}(\zeta_c) = 0$ を満たす次数 $k + \ell - 1$ 多項式。
- 全 ζ_c で 0 となる次数 $2k - 1$ 多項式 $p_{0,h}$ 。
- random vector $\mathbf{v}_h \in F^n$ 。
- query 列上の U^x, U^y, U^z, U^w の値。

その後、検査式が成立するよう blind row の query 値を解いて設定する。honest execution でもこれらの値は同じ affine subspace 上で一様なので、分布は完全に一致する。

Theorem 4.7

$e < (n - k)/4$ 、 $|F| \geq \ell + n$ 、 $m\ell > n_i + s$ 、 $k > \ell + t$ の下で、final protocol は perfect completeness、上記 soundness、perfect honest-verifier ZK を持つ。prover から verifier へ送る field element 数は

$$\sigma n + \sigma(k + \ell - 1) + \sigma(2k - 1),$$

verifier は oracle から t 個の列 symbol を読む。実用 argument では、最初の σn 長 vector も Merkle-committed oracle として処理され、必要位置だけが開かれるため、最終通信はこの生の和より小さくなる。

8 From ZKIPCP to ZK

8.1 対話型 argument への変換

情報理論的 ZKIPCP では verifier が oracle へ直接 access する。実際の network protocol では、prover が oracle symbol へ statistically hiding commitment を行い、その commitment 列を Merkle tree で圧縮する。以後の対話は ZKIPCP を模倣し、query された列だけを decommit する。

この変換では二つの binding が必要である。

1. leaf commitment が開示値を固定する。
2. Merkle root が leaf commitment 列を固定する。

どちらも CRH を black-box 利用して構成できる。honest-verifier ZK から malicious-verifier ZK への強化には、ZKPCP 文献の標準 compiler を用いる。

8.2 非対話型 argument への変換

各 prover oracle を Merkle root へ置換し、各 verifier challenge を transcript に対する random-oracle output から生成する。最終 proof は、

- ・ 各 round の Merkle root
- ・ prover の短い応答多項式または係数
- ・ query された列値
- ・ authentication paths

からなる。

BCS 変換により、元の IOP が ZK なら ROM 内で ZK が保たれる。soundness は元 IOP の通常 soundness だけでなく、state-restoration soundness で評価される。一般には round 数 $k(x)$ 、RO query 上限 T 、元 soundness $\epsilon(x)$ に対して

$$k(x)T\epsilon(x) + O(T^2 2^{-\kappa})$$

型の上界が得られる。

8.3 round-by-round soundness による Ligero 固有の改善

Ligero では bad transcript の状態を round ごとに分類できる。

- ・ 最初の algebraic challenge 後：ランダム線形結合が偶然 bad oracle を隠す error

$$\epsilon_1 = \frac{d+2}{|F|^\sigma}.$$

- ・ query challenge 後：誤り列を外す、または偽の多項式が query 点で一致する error

$$\epsilon_2 = \left(1 - \frac{e}{n}\right)^t + 2 \left(\frac{e+2k}{n}\right)^t.$$

悪意ある prover が前者の prefix を T_1 回、後者を T_2 回 RO へ query すると、成功確率は

$$T_1 \epsilon_1 + T_2 \epsilon_2 + T^2 2^{-\kappa} \leq T(\epsilon_1 + \epsilon_2) + T^2 2^{-\kappa}$$

で抑えられる。原論文は、 κ -bit の concrete security を狙う際、interactive soundness を約 $2^{-\kappa}$ 、RO output を約 2κ bits に設定する説明を与える。

注釈

これは古典 ROM の見積りである。量子 adversary が superposition query を行う QROM では、同じ係数・損失がそのまま成立するとは限らない。実装仕様では、domain separation、transcript serialization、hash output length、challenge bias removal を明示しなければならない。

8.4 Sublinear parameterization

最終通信量を大づかみに書くと

$$Comm \approx \underbrace{\sigma}_{\text{test}} \left(k + \underbrace{(k + \ell - 1)}_{\text{field elements}} + \underbrace{(2k - 1)}_{\text{Merkle authentication}} \right) \log |F| + t \log n \cdot h,$$

である。第一行は三 test の応答、第二行は開く列の field elements、第三行は Merkle authentication である。

改善解析に基づき $e = k$ 、 $n = 3k$ とすると $e < d/2$ であり、主要 soundness は

$$3 \left(\frac{2}{3} \right)^t + \frac{n+4}{|F|^\sigma}$$

型になる。したがって

$$t \approx \frac{K}{\log_2(3/2)}, \sigma \approx \frac{K}{\log_2 |F|}$$

と選ぶ。

k は ZK 条件 $k > \ell + t$ を満たし、FFT 実装上は 2 の冪に丸める。 $m \ell \gtrsim |C|$ なので、通信の支配項

$$\ell + t m \approx \ell + \frac{t|C|}{\ell}$$

を最小にする $\ell \simeq \sqrt{t|C|}$ が平方根通信を与える。

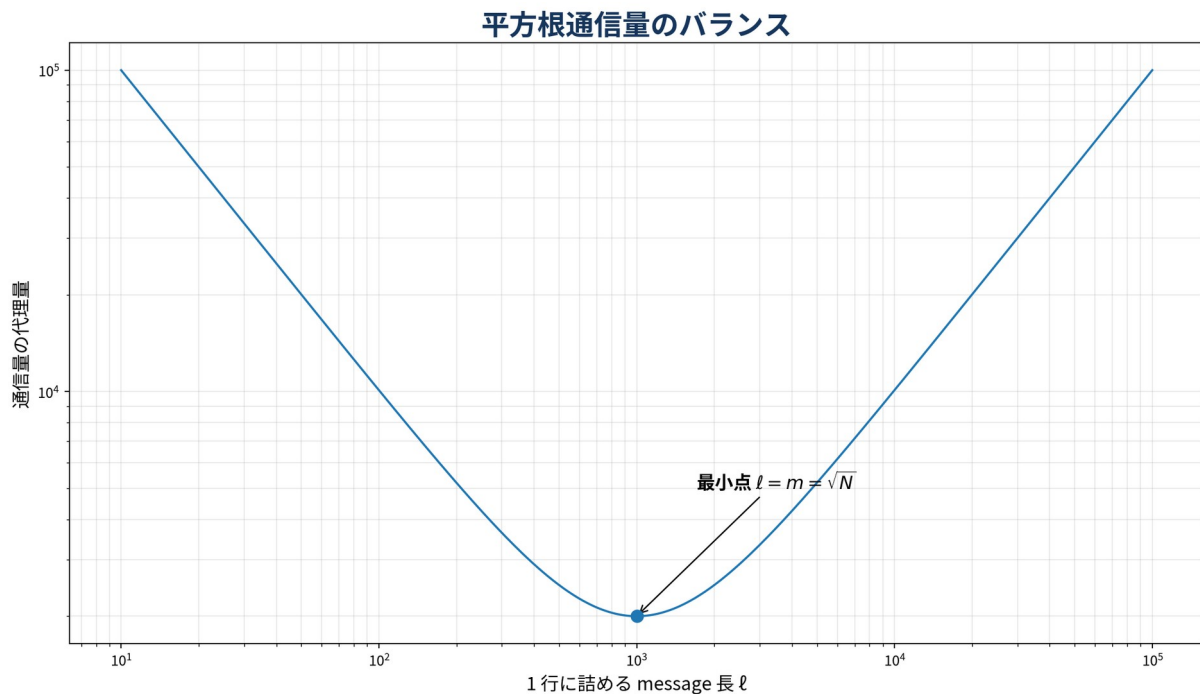


図9 row response 側の $O(\ell)$ と column opening 側の $O(t s / \ell)$ の均衡点が平方根を生む。

8.5 Multi-instance amortization

同一回路 C に対する N 個の witness を証明する場合、各 instance を行方向に束ね、制約 matrix や補間係数の計算を共有できる。原論文の漸近評価では、十分大きい field で総通信量は

$$O((N + \kappa|C|)|F|)$$

型となり、 $N = \Omega(\kappa^2)$ で一 instance 当たり通信が回路サイズを下回る。検証側も同一回路に依存する計算を一度だけ行い、総計

$$O(|C| \log |C| + N \log N)$$

へ amortize される。

Multi-instance amortization

同一回路 C に対する N 個の witness を「同じ位置ごと」に横方向へ束ねる

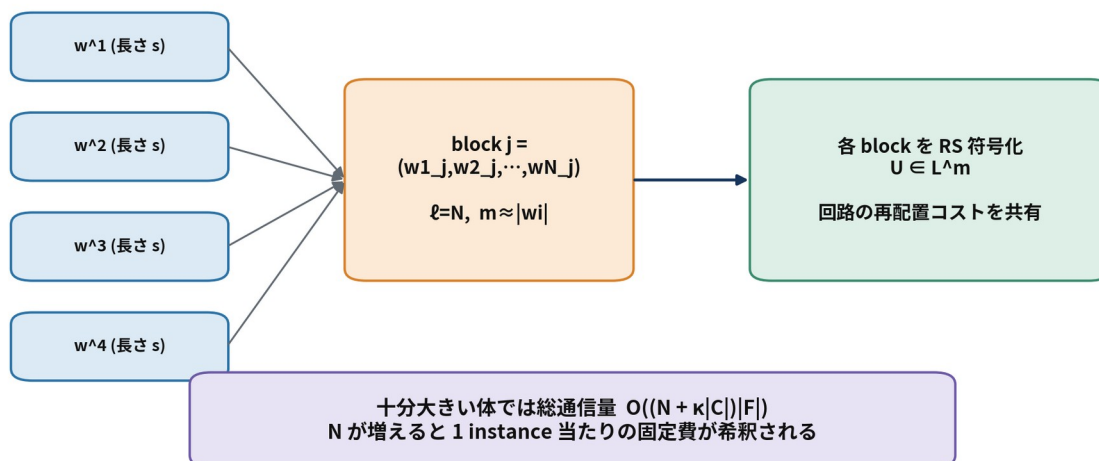


図 10 回路依存前処理と符号化・challenge を共有し、instance 数が増えるほど平均コストを下げる。

9 Implementation and Experimental Results

9.1 実装方針の翻訳

2022 年版実装は、prime field arithmetic と FFT を中心とする。Reed–Solomon encoding、補間、評価が prover cost を支配するため、NTT/FFT-friendly prime と高速多倍長演算が重要となる。論文実装は NFLlib を用い、30-bit prime field、SHA-256 による Merkle/Fiat-Shamir、Amazon EC2 c6i.32xlarge 相当の多数 core 環境で測定する。

prover 処理は回路トポロジーより witness length、すなわち gate 数に強く依存する。これは、配線が sparse matrix として一度構築された後、主要処理が matrix-vector 操作と RS encoding になるためである。

9.2 実装で効く最適化

- k, n を FFT-friendly な 2 の冪へ丸める。
- 三 test で同一 query columns を共有する。
- 反復 σ を extension field または並列 challenge としてまとめる。
- public vector a, b の encoding を再利用する。
- sparse matrix P_x, P_y, P_z, P_{add} を streaming 可能な形式で持つ。
- Merkle leaf に同一 column の全 field element を詰める。
- multi-core で row encoding と hash tree construction を並列化する。

9.3 proof length の改訂比較

原論文 Table 2 は、128-bit 設定の他方式データと改訂 Ligero の proof length を比較する。異なる field、statement 生成、machine を含むため厳密な apples-to-apples 比較ではないが、改訂 soundness 解析の効果を見る資料になる。

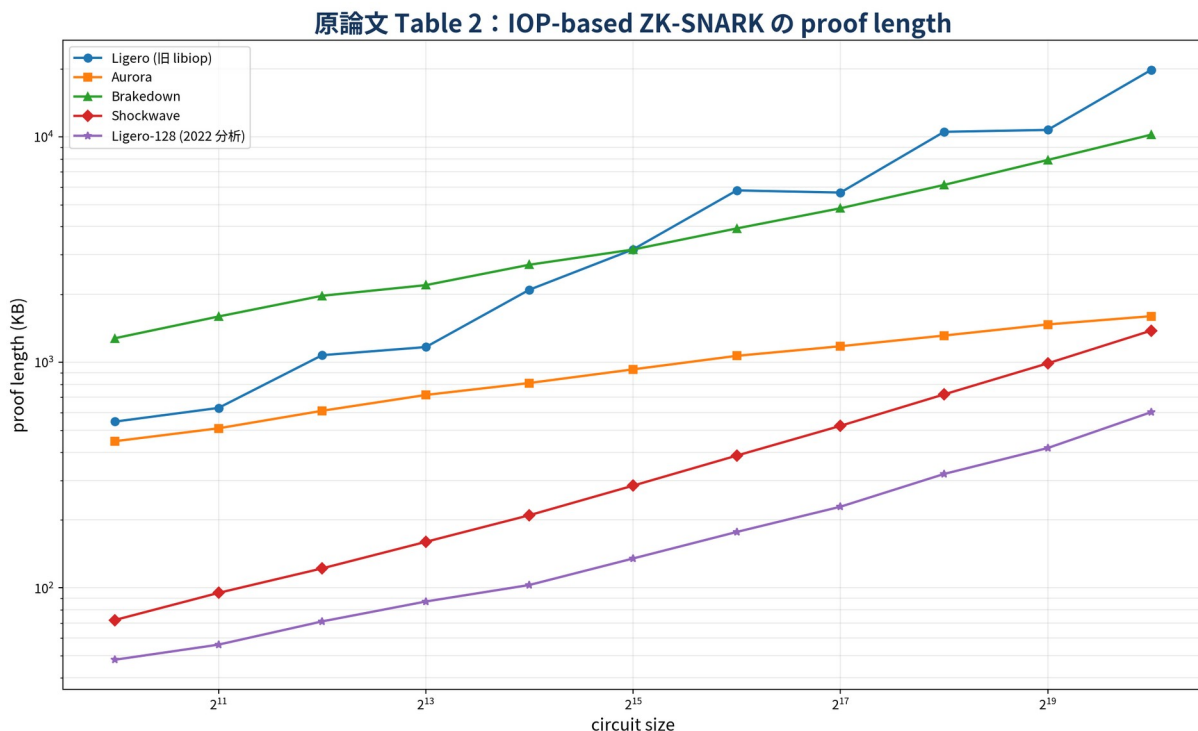


図 11 原論文 Table 2 を再可視化。改訂 Ligero-128 は 2^{10} から 2^{20} gate で約 48 KB から 602 KB と報告される。

重要な読み方は次である。

- ・ 旧 Ligero parameter は field/error bound が保守的で、proof が大きい。
- ・ $e < d/2$ 解析と joint soundness により code rate と query parameter を改善できる。
- ・ Aurora は polylog proof だが、当該小中規模では定数項により Ligero-128 が短い領域がある。
- ・ Shockwave は短い proof を示すが、prover/verifier trade-off と field 条件が異なる。

9.4 prover/verifier time

原論文 Table 3 では、回路サイズ 2^{20} から 2^{23} に対して、Ligero-128 prover が 4、5、7、11 秒、verifier が 3.8、4.7、6.6、10 秒と報告される。比較対象の Brakedown/Shockwave は別 machine であり、結果は傾向としてのみ読むべきである。

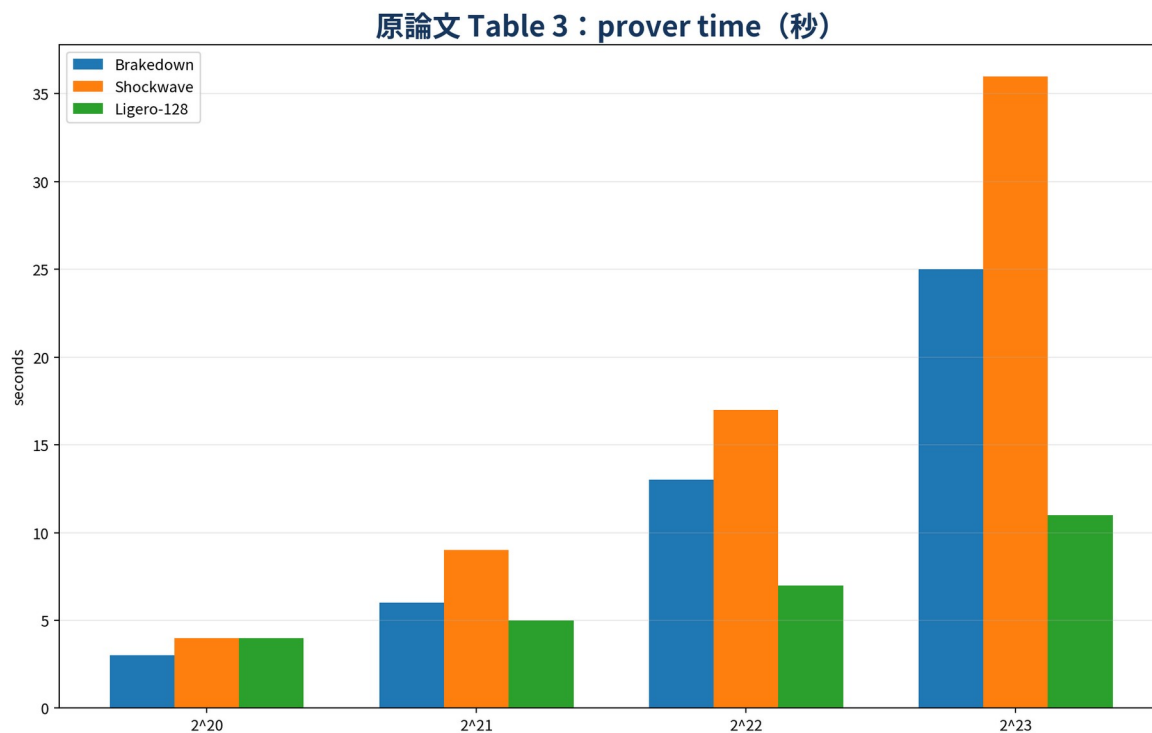


図 12 Table 3 の prover time。

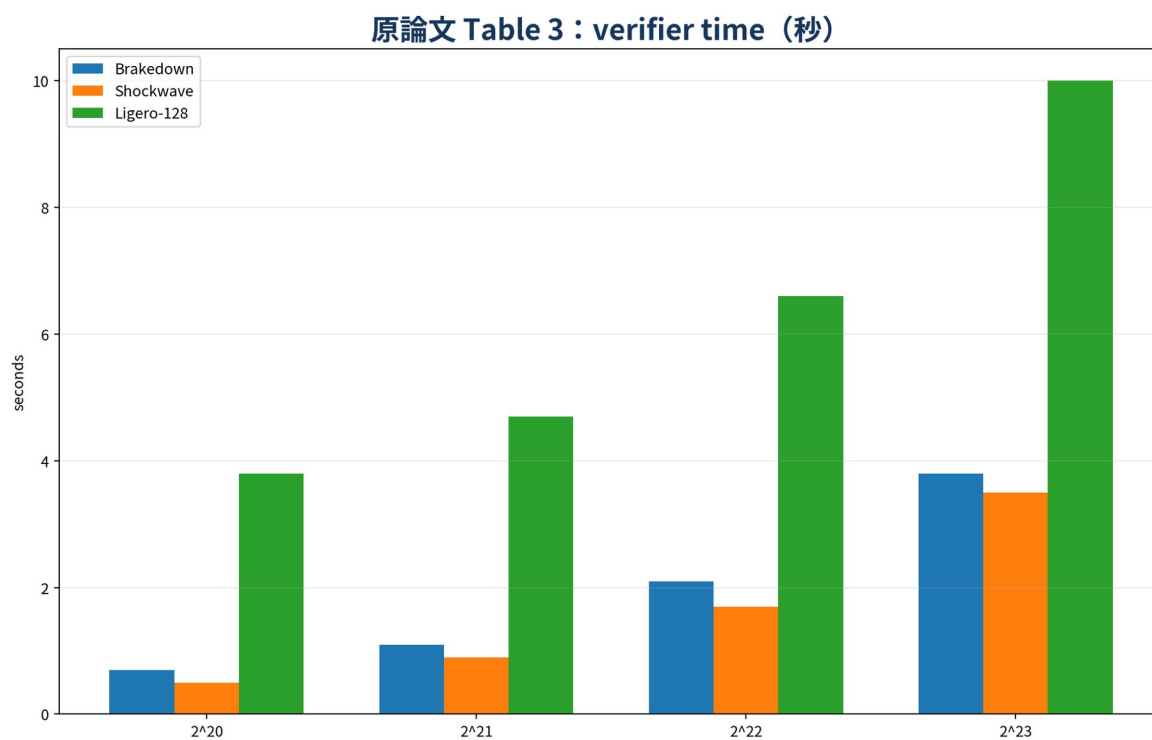


図 13 Table 3 の verifier time。Ligero は prover 側に強い一方、verifier は linear work と FFT 前処理のため重い。

9.5 SHA-256 preimage benchmark

40-bit soundness で約 35 KB という値は、Boolean 回路化、field embedding、当時の parameter selection に基づく。現代の 128-bit security、QROM margin、larger hash output、proof metadata を含めれば増加する。したがって、これは「Ligero が数十 KB 級になり得る」という歴史的 demonstration として扱う。

9.6 benchmark を比較するときの監査項目

1. security level は同じか。classical/quantum、statistical/computational を区別したか。
2. field element の bit 長は同じか。
3. circuit model は Boolean、arithmetic、R1CS、AIR のどれか。
4. circuit preprocessing/setup time を含むか。
5. hash/Merkle 生成と transcript serialization を含むか。
6. prover memory と disk spilling を含むか。
7. single-thread か multi-thread か。
8. proof size に public input、verification key、Merkle roots を含むか。
9. 同一回路 batch か single instance か。
10. soundness 解析は proved bound か conjectured/tuned parameter か。

10 Related Work and Open Problems

10.1 Constant computational overhead

Ligero の算術回路 prover overhead は RS encoding に由来する $O(\log|C|)$ 程度で、Boolean 回路では field embedding を含む polylogarithmic overhead となる。後続研究は large-field R1CS に対する linear-time prover を実現したが、Boolean 回路で negligible soundness を保ちながら constant overhead と sublinear query を同時に達成する問題は難しい。

Open Problem 1 の翻訳

サイズ s の Boolean circuit satisfiability に対し、soundness $2^{-\lambda}$ 、verifier query $\lambda \text{polylog}(s)$ 、prover circuit size

$$O(s) + \text{poly}(\lambda, \log s)$$

となる IOP を構成できるか。

10.2 Complexity-preserving IOP

多くの succinct proof は time をほぼ線形にできても、execution trace 全体を保持するため space が $\Omega(T)$ となる。RAM 計算が time T 、space $S \ll T$ で動くなら、理想は prover も $\tilde{O}(T)$ time、 $\tilde{O}(S)$ space、verifier は polylogarithmic time/space である。

Open Problem 2 の翻訳

universal RAM relation に対し、prover time/space、verifier time、communication のすべてを元計算の time/space に対して polylog overhead 以内に保つ IOP を構成できるか。

2022 年版は Bhadauria らの complexity-preserving IOP を進展として挙げるが、完全な polylog verifier ・ polylog communication にはなお距離がある。

10.3 Minimal assumptions

Kilian 以来、CRH があれば public-coin sublinear argument を構成できる。しかし one-way function が十分か、必要かは明らかでない。

Open Problem 3 の翻訳

NP 全体に対する sublinear argument を構成するための最小暗号仮定は何か。

10.4 本書から見た追加の研究課題

- interleaved RS test の list-decoding 域までの soundness 強化。
- Ligero 型平方根行列化と FRI/STIR/WHIR 型 low-degree test の hybrid。
- ZK randomized encoding の entropy を減らす parameter 最適化。
- streaming/space-bounded prover での Merkle tree construction。
- QROM で tight な Fiat-Shamir 解析。
- binary field、prime field、extension field の hardware-aware 自動選択。
- 同一回路 batch だけでなく、近縁回路 family の shared preprocessing。

11 Conclusions の注釈付き翻訳

論文は、NP に対して具体効率と sublinear communication を両立する ZK argument を設計・実装したと結論づける。計算量の中心は多項式評価・補間であるため、FFT 実装が性能を左右する。

prime field 版では integer arithmetic を効率化しやすい。一方、characteristic 2 では XOR が field addition となるため Boolean circuit に自然であるが、AND の auxiliary representation や FFT 実装の trade-off がある。

verifier は必要 query point だけが欲しいにもかかわらず、実装では全 domain FFT を行ってから抽出する部分がある。subset evaluation、GPU FFT、uniform circuit structure の利用により改善余地がある。最後に、bivariate polynomial PCP や別 MPC protocol を用いた軽量 sublinear ZK の具体効率を探索すべきだと述べる。

12 Appendix A–C の注釈付き翻訳

12.1 Appendix A : $e < d/3$ における affine-line 補題

目標は、affine line $\ell_{u,v}$ が RS code の半径 e 近傍球を多数横切ることではないと示すことである。
codeword を加えても距離構造は変わらないため、 u, v 自身の weight が高々 e の場合へ平行移動できる。

Case 1 : $|supp(u) \cup supp(v)| \leq e$

line 上の全点は zero codeword から高々 e なので、すべて近い。これは補題の第一分岐である。

Case 2 : support union が $e+1$ 以上

u, v の共通 support 各座標では、 $u_j + \alpha v_j = 0$ となる α は高々一つである。zero codeword の e -ball へ入る点は少数になる。

さらに nonzero codeword c の e -ball と line が交わると仮定すると、ある weight $\leq e$ の w について

$$c + w = u + \alpha v$$

となる。よって c は三つの weight $\leq e$ vector の和で weight $\leq 3e$ 、しかし nonzero RS codeword の weight は少なくとも $d > 3e$ で矛盾する。

この議論から、全点が近い場合を除き、近い点は高々 d 個という結論を得る。

12.2 Appendix B : generalized tests

B.1 parallel repetition of interleaved test

σ 個の独立な $r_h \in F^m$ を送り、 $w_h = r_h^\top U$ を返させる。同じ query set Q で全応答を検査する。field 由来 error は

$$\left(\frac{e+1}{|F|} \right)^\sigma$$

型へ減る。実装上は F^σ を extension field $\hat{F}$ とみなし、一つの challenge へまとめられる。

B.2 affine interleaved test

応答を $w = r^\top U + u^0$ とし、 u^0 をランダム codeword とする。これは linear subspace ではなく affine coset の membership を検査する。 u^0 の係数を固定 1 にすることで blind が 0 になる事象をなくす。

B.3 generalized affine interleaved test

parallel repetition と affine blinding を合成する。soundness amplification と perfect ZK を同時に得る。

B.4 generalized affine linear constraint test

σ 個の random compression を extension field 上でまとめ、zero-sum message を持つ blind row を加える。soundness は

$$|F|^{-\sigma} + \left(\frac{e+k+\ell}{n} \right)^t$$

型、改善解析では e を除いた項になる。

B.5 generalized quadratic constraint test

同様に zero-message blind row を加え、 σ 並列化する。soundness は

$$|F|^{-\sigma} + \left(\frac{e+2k}{n} \right)^t$$

型である。

12.3 Appendix C : joint soundness analysis

基本解析は三 test の error を独立に union bound するため、linear/quadratic test の query miss 項に誤り列数 e が重複して入る。Appendix C は interleaved test と同時に解析し、この重複を除く。

bad event を次のように置く。

- E_1 : U に e より多い誤り列がある。
- E_2 : prover が返した RS codeword $\mathbf{w}^i$ が正しい random combination $\mathbf{w}$ と異なる。
- E_3 : query set が誤り列を含む。

E_1 は RS proximity 解析、 E_2 は二符号語の一致点数、 E_3 は random coefficient が誤りを相殺する確率で抑える。結果として IRS test 関連 error は

$$\frac{n+1}{|F|} + \left(1 - \frac{e}{n} \right)^t$$

型になる。

E_1, E_2, E_3 が起きていないと条件付けければ、linear test で誤り列を query 候補に含める必要がなく、

$$\left(\frac{k+\ell}{n} \right)^t + \frac{1}{|F|}$$

となる。quadratic test も

$$\left(\frac{2k}{n} \right)^t + \frac{1}{|F|}$$

となる。

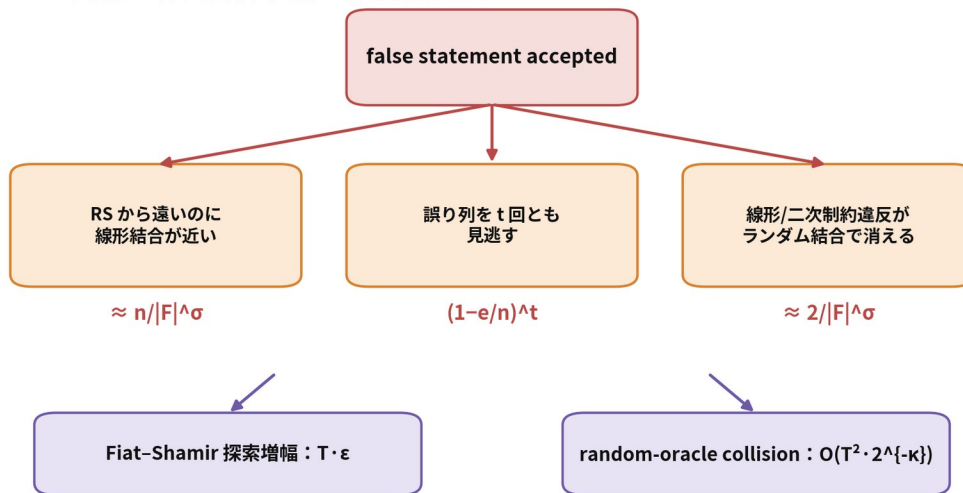
30-bit prime field と σ 反復を用いる最終的なまとめは概ね

$$\left(1 - \frac{e}{n} \right)^t + \left(\frac{k+\ell}{n} \right)^t + \left(\frac{2k}{n} \right)^t + \frac{n+3}{(2^{30})^\sigma}$$

である。

健全性誤差の分解

Ligero のパラメータ設計は、異なる失敗事象を別々に抑える作業である



最終的には union bound / round-by-round soundness で合成

図 14 符号 proximity、制約圧縮、column miss、hash binding という異なる error source を分けて管理する。

Part III 数学的仕組みの深掘り

13 なぜ平方根通信量になるのか

13.1 最小モデル

回路の extended witness 長を s とし、これを $m \times \ell$ 行列に配置する。

$$m\ell \geq s.$$

Ligero の通信コストを、定数・hash path・反復をいったん無視して分解する。

- ・ 応答多項式の係数数： $O(k + \ell + 2k) = O(k + \ell)$ 。
- ・ ZK 条件： $k > \ell + t$ なので、最小限では $k = \Theta(\ell + t)$ 。
- ・ query 列の開示：一列に $\Theta(m)$ field elements があり、 t 列で $\Theta(tm)$ 。

したがって

$$Comm(\ell) = \Theta(\ell + t) + \Theta\left(t \frac{s}{\ell}\right).$$

t を security parameter に依存する定数とみなせば、微分または AM-GM より

$$\ell^i = \Theta(\sqrt{ts}), m^i = \Theta\left(\sqrt{\frac{s}{t}}\right),$$

であり、

$$Comm(\ell^i) = \Theta(\sqrt{ts}).$$

$t = \Theta(\kappa)$ なら $\Theta(\sqrt{\kappa s})$ である。polylog factor は Merkle path、FFT-friendly rounding、field repetition から来る。

13.2 一般 MPC 変換との対応

抽象 MPC 変換では、party 数を n_p 、一 party view を s/n_p としたとき

$$Comm \approx n_p + t_p \frac{s}{n_p}.$$

直接構成の ℓ は receiver/global response 側、 m は server/local view 側に対応する。つまり平方根は、

- ・ global に送る「圧縮された全体情報」
- ・ local に開く「ランダム部分情報」

の積が元計算サイズに固定されるときの一般的均衡である。

13.3 Merkle path を入れた最適化

一列の field payload を $c_m m \log |F|$ 、authentication path を $h \log n$ とすると、

$$Comm(\ell) \approx c_1 \ell \log |F| + t \left(c_m \frac{s}{\ell} \log |F| + h \log n \right).$$

Merkle 項は ℓ にほぼ依存しないため、最適点は依然平方根型である。ただし小規模では hash path の固定費が支配し、sublinear の漸近優位が現れない。

13.4 回路トポロジーに依存しにくい理由

Ligero は、回路を層ごとに sumcheck するのではなく、全 wire 値を flat vector へ置き、selection matrix で配線を表す。したがって depth、fan-out、layer width は proof length に直接入らない。代わりに、

- gate 数
- input/witness 長
- sparse matrix の nonzero 数
- field embedding で増える auxiliary bit

が効く。これは deep/narrow circuit と wide/shallow circuit で proof parameter が大きく変わらないという長所であり、uniform structure を活かさないという弱点でもある。

14 soundness 証明の解剖

14.1 全体の論理

false statement に対する任意の oracle U^{ϵ} を考える。次の排反に分ける。

Far case

$$d(U^{\epsilon}, L^{4m}) > e.$$

この場合、interleaved test が reject する。失敗は、

1. random row combination が偶然 code 近傍へ落ちる。
2. t query が不整合列をすべて外す。

のいずれかである。

Near case

$$d(U^{\epsilon}, L^{4m}) \leq e.$$

$e < d/2$ なら最寄り codeword 行列 U は一意である。 U が復号する w, x, y, z が全回路制約を満たせば、元の circuit は satisfiable になり矛盾する。したがって、少なくとも linear または quadratic constraint が偽である。各 constraint test の soundness が適用される。

この二分法が、PCP/IOP で頻出する proximity + algebraic consistency の標準構造である。

14.2 interleaved test の代数幾何的直観

U^{ϵ} の row span L^{ϵ} を affine subspace とみなす。random r は L^{ϵ} の random point w^{ϵ} を選ぶ。bad oracle を見逃すには、 w^{ϵ} が RS code の小さな Hamming ball union へ入らねばならない。

RS code の minimum distance が大きいほど近傍球は互いに離れている。affine line が複数の ball を大量に横切ると、二つの近点の差から low-weight nonzero codeword ができ、minimum distance に反する。この「線と符号近傍の交差数」の制御が、 $1/|F|$ 、 $d/|F|$ 、 $n/|F|$ の field 項を生む。

14.3 linear constraint test の多項式恒等性

正しい q と偽の応答 q' は次数高々 $k+\ell-2$ である。 $q \neq q'$ なら差 $q-q'$ は非零多項式で、根は高々 $k+\ell-2$ 個である。よって、誤り列を除けば、偽応答が local consistency を通る列は高々 $k+\ell-2$ 個である。

このとき t query を replacement なしで選ぶ exact probability は

$$\frac{\binom{e+k+\ell-2}{t}}{\binom{n}{t}},$$

粗い上界が

$$\left(\frac{e+k+\ell}{n}\right)^t$$

である。証明長最適化では、この粗い上界を使うか、exact hypergeometric tail を使うかで concrete parameter が変わる。

14.4 quadratic test の次数

p_i^x, p_i^y は次数 k なので積は次数 $2k-1$ 。したがって偽の p_0' と正しい p_0 の一致点数は高々 $2k-2$ 。これが query miss 項 $(2k/n)^t$ の由来である。

14.5 random linear compression の error

偽の constraint residual vector を $\delta \neq 0$ とすると、random r に対して

$$\langle r, \delta \rangle = 0$$

となる確率は exactly $1/|F|$ である。 σ 独立反復なら $|F|^{-\sigma}$ 。これは Schwartz-Zippel より単純な、nonzero linear functional の kernel size に基づく。

14.6 union bound の保守性

basic proof は各 test の bad event を別々に上界評価して足す。そのため、同じ誤り列が interleaved、linear、quadratic の各項に重複計上される。Appendix C は共通 query set という構造を使い、interleaved test が通った条件下で constraint test を解析する。これは一般に **joint soundness analysis** と呼べる発想で、concrete proof system では大きな改善をもたらす。

15 完全零知識の数理

15.1 randomized Reed–Solomon encoding

message 値 $x_1, \dots, x_\ell$ を $\zeta_1, \dots, \zeta_\ell$ で固定する次数 k 多項式全体は、affine subspace

$$P_x = \{ p \in F_{\ell k}[X] : p(\zeta_c) = x_c, c \in [\ell] \}$$

をなす。dimension は $k - \ell$ である。

L 上の distinct query 点 $\eta_{j_1}, \dots, \eta_{j_t}$ への evaluation map

$$E_Q : P_x \rightarrow F^t, p \mapsto (p(\eta_{j_1}), \dots, p(\eta_{j_t}))$$

を考える。 η と ζ が disjoint で $k - \ell \geq t$ なら、この map は surjective である。したがって p を P_x から一様を選ぶと、 $E_Q(p)$ は F^t 上一様で、 x に依存しない。

この線形代数が $k > \ell + t$ 条件の本体である。

15.2 global response を隠す必要

局所 query が一様でも、 $r^\top U$ 全体を送ると witness の linear information を漏らす。そこで random codeword u^0 を加える。

$$v = r^\top U + u^0.$$

u^0 が適切な subcode/coset 上で一様なら、 v も同じ空間上で一様になり、 $r^\top U$ の shift を隠す。これは one-time pad の線形符号版である。

15.3 affine rather than linear blinding

もし

$$v = r^\top U + r_0 u^0$$

で r_0 も一様なら、 $r_0 = 0$ の確率 $1/|F|$ で blind が消える。 r_0 を nonzero から選ぶと soundness 解析が変わる。Ligero は係数を 1 に固定した affine combination を採用し、常に blind を残しつつ元の distance argument を一般化する。

15.4 simulator の構成

honest verifier の challenge と query set Q が既知とする。simulator は次の順に生成する。

1. verifier へ送る応答多項式を、必要な zero-sum/vanishing 条件を満たす空間から一様を選ぶ。
2. query 列の主要 oracle 値を一様を選ぶ。
3. 検査方程式を満たすよう blind row の query 値を解く。
4. 非 query 位置は view に現れないため生成不要。

real protocol でも、randomized encoding の surjectivity と blind codeword の一様性により、同じ変数が同じ affine space 上で一様である。よって statistical close ではなく identically distributed である。

15.5 malicious verifier への拡張

直接の Lemma 4.15 は honest verifier を扱う。一般 malicious verifier は challenge 分布を偏らせたり、commit 前後で adaptive query を行ったりする。ZKPCP/IOP compiler は、statistically hiding

commitment、coin-flipping、simulator-friendly commitment 等を用いてこれ进行处理する。非対話 ROM 版では、transcript hash が challenge を固定し、ZK simulator は random oracle programming を使う。

16 有限体 F_{17} の手計算例

16.1 message と RS 符号語

説明用に F_{17} 、message 点 $\zeta=(0,1,2)$ 、codeword 点 $\eta=(3,4,5,6,7,8)$ とする。

第一行 message を $(2,3,4)$ とする。補間多項式は

$$p_1(X)=2+X.$$

第二行 message を $(5,3,0)$ とすると

$$p_2(X)=5+7X+8X^2(mod\ 17).$$

各 η_j で評価すると

$$U_1=(5,6,7,8,9,10),$$

$$U_2=(13,8,2,12,4,12).$$

16.2 random linear combination

verifier が

$$r=(7,11)$$

を選ぶ。すると

$$w=7U_1+11U_2(mod\ 17)=(8,11,3,1,5,15).$$

対応する多項式は

$$q(X)=7p_1(X)+11p_2(X)=1+16X+3X^2(mod\ 17).$$

たとえば $X=3$ で

$$q(3)=1+48+27=76\equiv 8(mod\ 17),$$

行列側でも

$$7\cdot 5+11\cdot 13=178\equiv 8(mod\ 17).$$

random query が列 1 を開けば、この一致を確認できる。

有限体 F_{17} による小さな Test-Interleaved 例

2 行の message を 6 点で評価し、 $r=(7,11)$ で圧縮する

message 行列

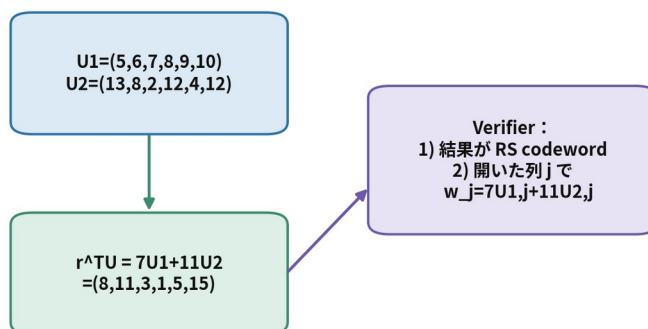
2	3	4
5	3	0

$\eta=3,\dots,8$ で評価

補間多項式

$$p_1(X)=2+X$$

$$p_2(X)=5+7X+8X^2$$



組合せ多項式は $1+16X+3X^2$ 。6 点の評価が上の w と一致する。

図 15 message 補間、RS 評価、ランダム行結合、query 整合性の数値例。

16.3 不正列がある場合

第二行の第 4 列を 12 から 13 に改ざんしたとする。w を正しく計算した prover は第 4 列で

$$7 \cdot 8 + 11 \cdot 13 \neq q(6)$$

となる。偽の w' を送れば、 $w' \in L$ 条件と他の query 列との整合を同時に満たさなければならない。誤り列が多数なら random query が高確率で当たり、誤りを相殺する random coefficient は field 全体のうち少数しかない。

16.4 randomized encoding の例

同じ message $(2, 3, 4)$ を持つ次数 5 多項式は

$$\begin{aligned} p_1'(X) &= p_1(X) + Z_\zeta(X)r(X), \\ Z_\zeta(X) &= X(X-1)(X-2), \deg r < 2 \end{aligned}$$

と書ける。 r を一様を選ぶと、message 点では $p_1' = p_1$ だが、disjoint な codeword 点での二つまでの評価は一様に randomize される。これが局所 ZK の具体像である。

17 parameter 設計の実務

17.1 入力

parameter generator が受け取るべきものは少なくとも次である。

- 回路 gate 数 s 、input/witness 長
- field F の bit 長と FFT 可能な subgroup size
- 目標 soundness $2^{-\lambda}$
- classical または quantum threat model
- hash output length h
- single/multi-instance、batch 数 N
- 最大 prover memory、thread 数、network bandwidth

17.2 基本手順

1. improved soundness を使えるなら $e=k$ 、 $n \approx 3k$ を初期候補とする。
2. column miss に対して

$$3(2/3)^t \leq 2^{-\lambda-\mu}$$

を満たす最小 t を選ぶ。 μ は union bound と Fiat-Shamir loss の余裕。 3. field error に対して

$$(n+c)|F|^{-\sigma} \leq 2^{-\lambda-\mu}$$

を満たす σ を選ぶ。 4. $m\ell \geq s+n_i$ 、 $k > \ell+t$ を満たし、通信量を最小化する m, ℓ を探索する。 5. k, n を FFT-friendly size へ切り上げる。 6. exact combinatorial probability

$$\binom{b}{t} / \binom{n}{t}$$

で再評価し、粗い power bound との差を詰める。 7. ROM loss、Merkle collision、commitment hiding error を加える。 8. 実測で FFT、hash、memory bandwidth の bottleneck を確認し、理論最小点から hardware 最小点へ調整する。

17.3 安全性 budget の分割例

総 error を

$$\epsilon_{total} \leq \epsilon_{IRS} + \epsilon_{lin} + \epsilon_{quad} + \epsilon_{FS} + \epsilon_{Merkle}$$

と分ける。128-bit target だから各項を機械的に 2^{-128} にする必要はなく、総和が target 以下となるよう配分する。ただし、 T 回の RO query により $T \epsilon_{IOP}$ へ増えるため、interactive layer には追加 margin が必要である。

17.4 よくある parameter 誤り

- $|F|$ と field bit length を混同する。
- σ を増やしたのに proof response を extension field element として圧縮せず、そのまま線形増加させる。
- ZK 条件 $k > \ell+t$ を soundness 条件だけで置き換える。
- $e < d/2$ を満たさず unique nearest codeword を仮定する。
- Merkle leaf packing を変えたのに query symbol 数を更新しない。
- Fiat-Shamir challenge を field へ mod reduction し、bias を無視する。

- batch で shared challenge を使う際、random linear combination の domain separation をしない。

18 ROM・Fiat-Shamir・実装安全性

18.1 transcript の canonicalization

proof parser の曖昧性は soundness proof の外側で攻撃面になる。hash input には以下を固定長または prefix-free に encode する。

- protocol/version identifier
- field identifier と modulus
- circuit digest
- round number と message type
- Merkle root
- vector length、matrix dimension、parameter tuple
- public input
- prior challenge と response

18.2 domain separation

同じ hash 関数から、

- row-combination coefficients
- linear constraint coefficients
- quadratic coefficients
- query indices
- Merkle node hash

を生成する場合、用途ごとに異なる label を入れる。異なる random variable を同じ digest slice で使い回すべきではない。

18.3 field sampling

hash digest を単純に $\text{mod } p$ すると bias が出る。 p が 2 の冪に近ければ小さいが、形式証明では rejection sampling または wide reduction を使う。zero challenge を除外する必要がある場合は、除外後分布を soundness 解析へ反映する。

18.4 query index の重複

replacement ありで t 回選ぶか、distinct subset として選ぶかで probability が異なる。原論文の一部表現は repetition を許す一般変換と subset sampling を使う直接 test が混在する。実装仕様はどちらかを固定し、proof parser と parameter generator を一致させる。

18.5 Merkle tree details

- leaf hash と internal node hash を domain separate する。
- left/right order を hash へ含める。
- variable-length leaf を曖昧なく serialize する。
- tree が 2 の冪でない場合の padding rule を固定する。
- duplicate query の authentication path を deduplicate しても、検証意味を変えないことを確認する。
- parallel construction で nondeterministic ordering を許さない。

18.6 QROM caveat

ハッシュ出力を 256 bits にしただけで「128-bit post-quantum soundness」が自動的に得られるわけではない。Grover 型加速、quantum collision search、QROM Fiat-Shamir reduction の loss を別々に考える。長期保存 proof では、hash agility と versioning を持たせることが望ましい。

Part IV Aurora・ZK-STARK・他の汎用 ZKP との比較

19 Ligero・Aurora・2018 ZK-STARK の共通基盤

19.1 共通する設計思想

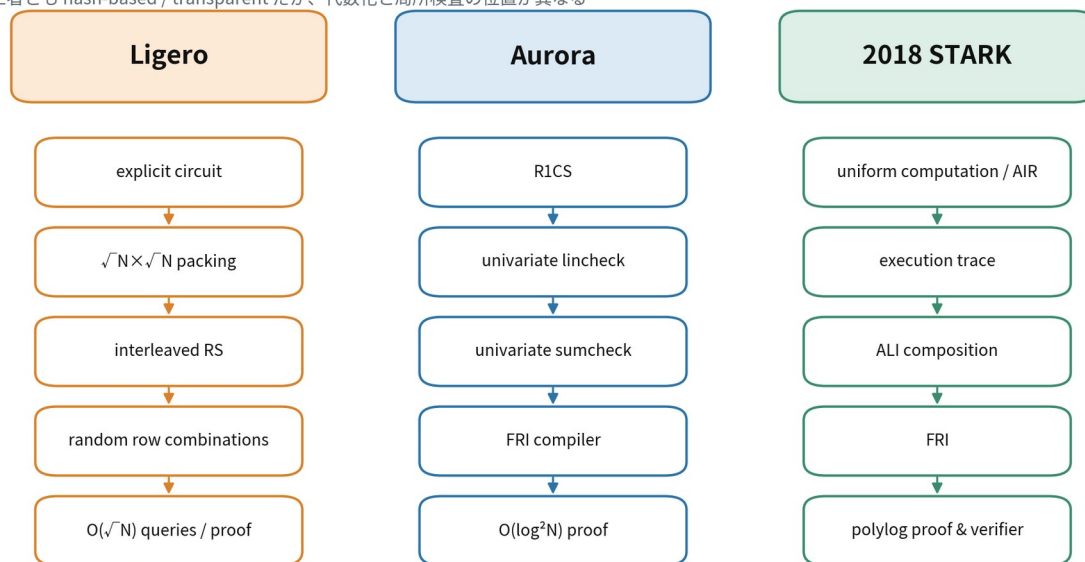
三方式はいずれも、次の層を共有する。

1. 計算を有限体上の低次数多項式関係へ arithmetize する。
2. witness または execution trace を Reed-Solomon 系 codeword として oracle 化する。
3. random linear combination で多数の関係を少数へ圧縮する。
4. oracle 全体は読まず、少数位置だけ query する。
5. Merkle tree で oracle を commit し、public-coin IOP/IPCP を Fiat-Shamir で非対話化する。
6. pairing や structured CRS を使わず、hash 中心の透明 argument を得る。

違いは、どの計算表現を起点にし、どの低次数性・制約性を、何 round で検査するかにある。

Ligero・Aurora・STARK の設計思想

三者とも hash-based / transparent だが、代数化と局所検査の位置が異なる



Ligero：定数と prover の軽さ / Aurora：明示 R1CS の短い proof / STARK：uniform 計算の succinct verifier

図 16 三方式の arithmetization、IOP core、proof scaling、適用領域の違い。

19.2 情報理論的 protocol の漸近比較

方式	起点	round	oracle proof length	query	prover	verifier (明示回路換算)
Ligero	算術/ Boolean circuit	2 程度	$O(N)$	$O(\sqrt{N})$	$O(N \log N)$	$O(N)$
Aurora	R1CS	$O(\log N)$	$O(N)$	$O(\log N)$	$O(N \log N)$	$O(N)$
2018 STARK	AIR/ bounded halting	$O(\log N)$	$O(N \log N)$	$O(\log N)$	$O(N \log^2 N)$ (一般回路換算)	$O(N)$ (明示回路換算)

この表は Aurora 論文の比較軸に沿う。STARK を uniform program として使う本来の設定では、verifier は program size と $\text{polylog}(T)$ に依存し、execution time T 全体を読む必要がない。明示的な N -gate circuit を program へ変換すると、program 記述自体が $\Theta(N)$ なので線形下限が戻る。

19.3 proof size の違い

Ligero

query する列数が $\Theta(\sqrt{N})$ なので、Merkle opening を含む argument size は

$$\tilde{O}(\sqrt{N}).$$

Aurora

univariate sumcheck で linear relation を対数 query へ落とし、FRI 系 low-degree test を使う。最終 argument は

$$O_\lambda(\log^2 N)$$

である。

STARK

AIR/ALI から二つの RS proximity problem へ落とし、FRI で検査する。proof size は同様に polylogarithmic だが、execution trace routing、composition polynomial、複数 oracle、認証 path が定数を増やす。

19.4 arithmetization の違い

Ligero：flat extended witness

全 wire 値を一つの vector へ並べる。配線は sparse selection matrix、乗算は Hadamard product で表す。depth に依存しないが、回路記述を明示的に処理する。

Aurora：R1CS

$$(A z) \circ (B z) = C z$$

を直接対象とする。三つの lincheck と一つの rowcheck へ分解し、univariate sumcheck で matrix relation を検査する。R1CS compiler との親和性が高い。

STARK：AIR

state vector x_t と次状態 x_{t+1} の local transition

$$P_i(x_t, x_{t+1}) = 0$$

を trace 全体で検査する。uniform、deep、narrow、長時間計算に強い。memory consistency には APR/routing が必要となる。

19.5 低次数検査の違い

Ligero

random row combination そのものを RS 符号語として送らせ、全体 membership を verifier が補間/評価で確認する。query 数は平方根だが protocol が単純で round が少ない。

Aurora

RS-encoded IOP を一般 IOP へ compile する際に FRI IOPP を使う。univariate sumcheck が「sum-zero subcode」への proximity 問題を作る。

STARK

FRI は protocol の中心であり、execution trace LDE と composition polynomial の低次数性を多段 folding で確認する。

19.6 ZK 技法の違い

方式	局所 query の遮蔽	global algebraic message の遮蔽
Ligero	同一 message を持つ random RS codeword	affine blind row
Aurora	degree slack による bounded independence	random self-reduction $cf+q$
STARK	randomized LDE / masking polynomial	ALI composition への randomization

三方式とも「codeword に余剰自由度を入れ、query 数以下の評価を一様化する」という同じ線形代数原理を使う。違いは、protocol 内で division/remainder や composition を行ったとき randomness が消えないよう、self-reduction または blind oracle を追加する点である。

20 三方式の強み・弱み

20.1 Ligero の強み

1. 構成が単純：RS encoding、random linear combination、少数列 opening の組合せで説明できる。
2. **prover が軽い**：複雑な routing network や多段 FRI を避け、主に FFT と sparse linear algebra。
3. **小中規模で固定費が小さい**：polylog proof 方式の Merkle root、folding oracle、複数 round の固定費を避ける。
4. **回路トポロジー非依存**：depth や width ではなく gate 数に主として依存。
5. **multi-instance に強い**：同一回路で matrix 処理と encoding を共有できる。
6. **transparent・hash-based**：structured setup なし、対称鍵中心。

20.2 Ligero の弱み

1. **proof が平方根**：百万から十億 constraint へ拡大すると、polylog proof との差が大きくなる。
2. **verifier が線形**：明示回路処理と多項式評価が必要。
3. **再帰に不向き**：hash/Merkle と多数 field element opening を別 proof 内で検証する cost が大きい。
4. **field element が大きい**：Boolean 計算でも prime field へ埋めると bit 効率を失う。
5. **bandwidth-sensitive**：on-chain や mobile network では proof size が支配的。
6. **ZK 条件による rate 低下**： $k > \ell + t$ の degree slack が code rate を下げる。

20.3 Aurora の強み・弱み

強み：明示 R1CS に対して proof が $O(\log^2 N)$ 、prover は $O(N \log N)$ 、transparent、plausibly PQ。univariate sumcheck により routing なしで linear relation を検査する。R1CS compiler との接続が明確。

弱み：verifier は R1CS matrix を読むため $O(N)$ 。FRI と複数 oracle の固定費があり、小規模では Ligero より重くなり得る。univariate sumcheck と domain 設計が Ligero より複雑。

20.4 2018 ZK-STARK の強み・弱み

強み：program と time bound で表した uniform computation に対して、verifier は execution time に polylogarithmic。deep/narrow sequential computation、rollup/zkVM 型の trace に自然。FRI による透明・PQ 志向の polylog proof。

弱み：AIR 設計、memory consistency、composition degree、blowup factor、FRI parameter が複雑。proof と prover memory の定数が大きい。明示回路に単純変換すると routing overhead で Aurora/Ligero に不利。

20.5 選択の要約

- ・ 同一の大きめ回路を多数回、**prover time 優先**：Ligero が有力。
- ・ 明示 R1CS、**proof bandwidth 優先**、**verifier 線形**を許容：Aurora 系。
- ・ 長時間 program execution、**verifier を time から切り離す**：STARK 系。
- ・ 極小 proof と高速 on-chain verification、**trusted/universal SRS**を許容：pairing-based SNARK。
- ・ 逐次計算の再帰・IVC：Nova/HyperNova 系。

21 他の汎用 ZKP との比較

21.1 比較軸

汎用 ZKP の優劣は単一指標で決まらない。少なくとも以下を分ける。

- setup：circuit-specific / universal / updatable / transparent
- assumption：pairing、DLP、hidden-order、lattice、hash/RO
- post-quantum：既知量子 algorithm への耐性
- arithmetization：R1CS、PLONKish、multilinear、AIR、Boolean
- prover time・memory
- verifier time
- proof size
- recursion/aggregation 適性
- field flexibility
- implementation maturity

21.2 Groth16

Groth16 は QAP/R1CS を pairing へ写し、proof を三つの group element に圧縮する。verifier は定数回の pairing と public input に比例する multi-scalar multiplication を行う。

Ligero より強い点：proof が数百 bytes、検証が極めて速い。on-chain verification に適する。

Ligero より弱い点：circuit-specific trusted setup、toxic waste、pairing/DLP に依存し量子耐性がない。prover は大規模 MSM/FFT を行う。

21.3 PLONK・Marlin 系

universal/updatable SRS と polynomial commitment を使い、多数回路を一つの最大 degree setup で扱う。PLONKish arithmetization は custom gate、permutation、lookup へ拡張しやすい。

強み：proof は小さく、verification も短い。compiler/tooling が豊富。

弱み：KZG 版は pairing と structured SRS に依存し、post-quantum ではない。IPA 版へ変えたと transparent にできるが proof/verifier trade-off が変わる。

21.4 Halo2 / IPA 系

inner-product argument 型 polynomial commitment を使う構成では trusted setup を避けられるが、楕円曲線 DLP に依存する。recursive composition と cycle-friendly curve の設計に強い。

Ligero より強い点：proof が対数的、再帰・aggregation に適する、PLONKish custom gate/lookup が使いやすい。

Ligero より弱い点：量子耐性がなく、prover は MSM 中心。bitwise 計算では non-native field cost が出る。

21.5 Bulletproofs

Pedersen commitment と recursive inner-product argument により、trusted setup なしで proof を $O(\log N)$ group elements にする。range proof の batch に特に強い。

Ligero より強い点：小さな proof、range proof の実績、setup 不要。

弱い点：DLP に依存し非 PQ。一般回路では verifier の group operation が線形で、prover も高価な group arithmetic を行う。

21.6 Spartan

R1CS を multilinear extension と sumcheck へ落とし、multilinear polynomial commitment と組み合わせる。情報理論的 core は単純で、sparse R1CS を活用できる。

Ligero より強い点：proof/query を対数化しやすく、multilinear representation が R1CS matrix へ自然。

弱い点：最終特性は polynomial commitment に依存する。DLP 版は非 PQ、hash/code 版は proof・verifier trade-off が変わる。preprocessing の有無も variant で異なる。

21.7 Brakedown と Shockwave

Brakedown は tensor/linear code 型 commitment により、arbitrary field 上で linear-time prover を目指す transparent post-quantum SNARK for R1CS である。proof/verifier は sublinear で、field-agnostic 性が大きな特徴である。Shockwave は RS code を使う variant として、proof size と verification を改善する代わりに linear-time 性との trade-off を持つ。

Ligero より強い点：linear-time prover、arbitrary field、R1CS との直接接続。

Ligero より弱い点：code construction と commitment layer が複雑で、proof が大きい parameter 領域もある。原論文 Table 2/3 では改訂 Ligero が小中規模で競争力を示すが、測定条件は異なる。

21.8 Nova・SuperNova・HyperNova

Nova 系は、step circuit の satisfaction を **folding scheme** で逐次 accumulate する incrementally verifiable computation (IVC) である。一 step ごとに proof state を更新し、最後に小さな SNARK へ compress する。

Ligero より強い点：長い逐次計算、再帰、proof-carrying data、unknown number of steps に強い。prover memory を step-local にできる。

Ligero より弱い点：標準構成は楕円曲線/DLP に依存し非 PQ。最終 ZK・compression は採用 PCS/SNARK に依存する。単発の flat circuit では folding の固定費が不要な Ligero が単純。

HyperNova は複数 step circuit、higher-arity folding、customizable constraint system へ拡張する。零知識は専用 masking を含む variant で提供される。

21.9 Binius

Binius は binary tower field 上での polynomial IOP/commitment を使い、bitwise computation を大 prime field へ無理に埋めない。hash、bit operation、lookup-heavy workload に適する。

Ligero より強い点：bit-level arithmetization の密度、binary field hardware との親和性、polylog proof architecture。

Ligero より弱い点：tower field・multilinear protocol・PCS が複雑で、設計と実装の成熟度は pairing/PLONKish より新しい。Ligero のような単純な平方根行列検査とは理解・監査コストが異なる。

21.10 STIR・WHIR

STIR と WHIR は、Reed-Solomon proximity testing / polynomial commitment を改善する新しい IOPP 系 building block である。STIR は FRI より低 query を狙い、WHIR は constrained RS code と sumcheck を統合して高速 verifier・小 proof を目指す。

これらは Ligero の直接代替というより、Aurora/STARK/Spartan 系の low-degree/PCS layer を更新する部品である。ただし、Ligero の interleaved test を多段 proximity proof へ置き換える hybrid は研究価値がある。

21.11 格子ベース sublinear R1CS argument

格子仮定に基づき、proof size が witness の平方根に比例する R1CS argument も提案されている。2022 年の実用方式は大規模 instance で Ligero より 2–3 倍短い proof を報告する。

強み：standard-model 寄りの post-quantum assumption、transparent、verification の改善可能性。

弱み：module-LWE/SIS parameter、rejection/rounding、large integer/vector arithmetic が複雑。hash-based Ligero より仮定と実装 attack surface が広い。

21.12 総合比較表

系統	setup	主仮定	PQ	proof scaling	verifier	得意領域
Ligero	transparent	CRH/RO	plausible	$\tilde{O}(\sqrt{N})$	$O(N)$	単純、batch、軽 prover
Aurora	transparent	CRH/RO	plausible	$O(\log^2 N)$	$O(N)$	explicit R1CS、小帯域
STARK	transparent	CRH/RO	plausible	polylog	program size + polylog T	uniform long computation
Groth16	circuit-specific	pairing/KEA 系	no	constant	constant + public input	最小 proof、on-chain
PLONK/Marlin-KZG	universal SRS	pairing	no	constant/polylog-small	small	general circuits、lookup
Halo2/IPA	transparent	DLP	no	polylog	polylog-small	recursion、PLONKish
Bulletproofs	transparent	DLP	no	$O(\log N)$	$O(N)$ group ops	range proof、batch
Spartan	PCS 依存	DLP/hash/code 等	variant	polylog	variant	sparse R1CS、sumcheck
Brakedown	transparent	hash/code	plausible	sublinear	sublinear	linear prover、arbitrary field
Nova/HyperNova	PCS 依存	DLP 等	usually no	recursive/constant state	incremental	IVC、sequential recursion
Binius	transparent	hash/code	plausible	polylog	small	binary/bitwise computation
lattice sublinear	transparent	lattice	yes 候補	$\tilde{O}(\sqrt{N})$	sublinear/variant	standard lattice assumption

表の注意

「constant」「polylog」は asymptotic class を示し、具体 proof size の順序ではない。setup を透明と呼んでも、curve parameters や hash function の合意は必要である。PQ 欄の“plausible”は、完全な QROM end-to-end proof と同義ではない。

22 用途別の選択指針

どの proof family を選ぶか

相対評価。実測値ではなく、設計上の傾向を示す

要件	Ligero	Aurora	STARK	Groth16/PLONK	Nova系
透明・PQを最優先	◎	◎	◎	△/×	×
proof を極小化	△	○	○	◎	○
moderate circuit の prover	◎	○	○	○	—
uniform 長時間計算	△	△	◎	○	◎
同一回路の多数 batch	◎	○	○	◎	◎
再帰・IVC	△	△	○	◎	◎

図 17 setup、proof bandwidth、uniformity、recursion、PQ 要件による概略選択。

22.1 confidential batch audit

同一検証回路で多数の record を処理し、proof 生成を server 側で行い、proof が数百 KB でも許容されるなら Ligero が合う。multi-instance amortization と回路トポロジー非依存性が効く。

例：KYC rule、private compliance、同一 ML inference rule、database integrity、集計関数の反復。

22.2 blockchain rollout

on-chain calldata と verification gas が最重要なら、Groth16/PLONK/KZG 系または再帰圧縮された STARK が通常有利である。Ligero proof をそのまま chain へ載せると平方根帯域と hash verification が重い。

ただし、off-chain committee 間の transparent audit、data availability とは別経路の proof、PQ migration study では Ligero 型が比較対象になる。

22.3 zkVM・長時間 program

program が小さく execution trace が長い場合、STARK/AIR または folding IVC が自然である。Ligero へ flat circuit 展開すると verifier が巨大回路を読むため、uniformity の利点を失う。

22.4 range proof・asset conservation

小規模 range proof や confidential transaction では Bulletproofs/PLONKish lookup が優位。Ligero は一般性の代わりに固定費と proof が大きい。

22.5 post-quantum long-term archive

trusted setup を避け、proof を長期保存し、将来の量子脅威を考える場合、hash-based STARK/Aurora/Ligero/Binius 系が候補になる。選択は proof size、verification software の持続性、hash agility、QROM security analysis で決める。

Part V 応用・実装・監査

23 Ligero の応用

23.1 outsourced computation

client が秘密入力 w を server へ渡して計算し、結果 y と proof を受け取る。statement を

$$\exists w : C(x, w) = y$$

とする。Ligero は、server が公開鍵群演算を大量に行わず、FFT と hash 中心で proof を生成できる。proof を network 越しに返すため、計算が十分大きいほど sublinear communication の意味が出る。

適する条件は次である。

- circuit が数百万 gate 以上。
- client 側で回路前処理 $O(s)$ を許容する、または同一回路で amortize できる。
- proof bandwidth が数十 KB から数 MB でも許容される。
- setup ceremony を避けたい。

23.2 confidential database audit

database D から統計量 $f(D)$ を計算し、 D を隠したまま正しさを証明する。commitment $c = H(D)$ を public input へ入れ、回路内で

$$H(D) = c, f(D) = y$$

を検査する。

Ligero の回路トポロジー非依存性は、record ごとに同じ predicate を適用する batch に向く。multi-instance として record batch を束ねるか、一つの大回路として行列化する。hash preimage circuit が大きい場合、proof system 内部の hash と statement 内の hash を区別する。前者は native SHA-256 でよいが、後者は field/Boolean circuit として証明する必要がある。

23.3 private compliance と policy proof

例として、秘密 transaction 列 $T_1, \dots, T_N$ が次を満たすことを証明する。

$$\begin{aligned} \sum_i amount(T_i) &= A, \\ \forall i : KYC(T_i) &= 1, \\ \forall i : country(T_i) &\notin B. \end{aligned}$$

同一 predicate を多数回評価するため、Ligero の batch 性が活きる。ただし、lookup-heavy policy は PLONKish lookup/Binius の方が arithmetization 効率に優れる場合がある。

23.4 machine-learning inference

固定 model M と秘密 input x に対して $M(x) = y$ を証明する。Ligero は dense linear algebra をそのまま効率化する専用 protocol ではないが、quantized Boolean/arithmetic circuit を flat に処理できる。

有利な場合：

- 同一 model で多数 inference を batch する。
- proof bandwidth より prover simplicity を優先する。
- model update ごとの trusted setup を避ける。

不利な場合：

- convolution/matrix multiplication 専用 sumcheck を使える。
- lookup/activation が支配的。
- recursive aggregation が必須。

23.5 cryptographic relation

SHA-256 preimage、signature verification、Merkle membership、key derivation などの Boolean circuit を証明できる。論文の SHA-256 benchmark はこの代表である。

ただし、proof system の外部 hash として SHA-256 を使うことと、SHA-256 回路の preimage を証明することは異なる。外部 hash は native 実行、内部 hash は gate 展開される。

23.6 proof of reserves / liabilities

秘密 liability list を commit し、総額、non-negativity、重複排除、public reserve との関係を証明する。Ligero は一括 batch へ自然だが、range constraints と set membership の回路 cost が大きい。現代実装では lookup argument や specialized range proof との hybrid が有利である。

23.7 MPC-in-the-head signature との概念的接続

Ligero の出発点は MPC-to-ZK であり、Fiat-Shamir により signature of knowledge へ応用する設計思想を共有する。ただし、Picnic 等の signature scheme は小さな Boolean relation と signature size に特化した MPC protocol を使う。汎用 Ligero をそのまま signature へ使うより、interleaved code test や affine blinding の技法を専用 MPC-in-the-head へ移す方が自然である。

24 実装アーキテクチャ

24.1 コンポーネント分離

安全な実装は次の module へ分ける。

1. **circuit frontend** : gate list、wire indexing、public/private input。
2. **constraint compiler** : P_x, P_y, P_z, P_{add} と Booleanity constraints。
3. **parameter generator** : $m, \ell, k, n, e, t, \sigma, F$ 。
4. **RS engine** : FFT/IFFT、coset evaluation、randomized encoding。
5. **prover algebra** : 三 test の response polynomial。
6. **oracle store** : column-major layout、streaming、memory mapping。
7. **Merkle engine** : leaf packing、parallel hashing、path pruning。
8. **transcript** : domain-separated Fiat-Shamir。
9. **verifier** : parser、root verification、field checks、constraint preprocessing。
10. **test harness** : property test、differential test、malformed proof corpus。

24.2 data layout

prover は row-wise FFT と column-wise Merkle construction の両方を行う。naive layout では transpose が memory bandwidth を支配する。

推奨設計：

- encoding 中は row-major buffer。
- block transpose して column-major oracle store へ書く。
- Merkle leaf は固定数 column を chunk 化し、cache line に合わせる。
- U^w, U^x, U^y, U^z と blind rows を同一 column block へ interleave する。
- query response は zero-copy slice で serialize する。

24.3 streaming prover

oracle 全体 $O(mn)$ を RAM に置けない場合、二 pass 以上で処理する。

1. extended witness を生成し disk/stream へ書く。
2. row block ごとに RS encode し、column shard へ転置書込み。
3. column shard から Merkle leaves と tree nodes を構築する。
4. challenge 確定後、必要な response polynomial を再計算または checkpoint から復元する。
5. query 列だけを read back する。

Fiat-Shamir では challenge が root に依存するため、root 生成前に query 位置は分からない。したがって、oracle の再読可能性または query 用 index が必要である。

24.4 parallelization

- 各 row の FFT は独立。
- sparse matrix-vector product は gate range で partition できる。
- Merkle leaf hash と各 level hash は parallel。
- σ 反復の response polynomial は parallel。

- verifier の query path と局所式検査も parallel。

ただし、transcript hash と round dependency は serial critical path を作る。NUMA 環境では row partition と column transpose の memory placement が重要である。

24.5 GPU 利用

GPU に適するのは large batched FFT、field element-wise product、Merkle hashing である。転送 cost を抑えるには、oracle 生成から Merkle leaf まで GPU 内に保持する。30-bit prime は 64-bit multiply/reduction で扱いやすいが、GPU ごとに最適 modulus が異なる。

24.6 verifier preprocessing

明示回路に対する linear-time verifier cost の多くは、 $r^T A$ 、selection polynomial、public encoding の構築である。同一回路を繰り返す場合、

- sparse matrix indexing
- FFT twiddle factors
- evaluation domain
- Merkle tree shape
- public constant encoding

を cache できる。ただし challenge 依存 vector 自体は毎 proof 再計算する。

25 security audit checklist

25.1 algebraic correctness

- modulus が prime であること、または extension field 定義が irreducible であること。
- evaluation points η_i が distinct。
- message points ζ_c が distinct かつ η と disjoint。
- n, k, ℓ, t, e が全定理条件を満たす。
- degree bound に off-by-one がない： ℓk 、 $\ell k + \ell - 1$ 、 $\ell 2k - 1$ 。
- Z_ζ の評価と zero-message condition が一致。
- Boolean embedding で field wraparound が起きない。
- public input wire が witness で上書きされない。

25.2 randomized encoding

- random coefficients が CSPRNG から均一に生成される。
- query bound と degree slack の関係 $k - \ell \geq t$ を満たす。
- 同じ random polynomial を複数 proof で再利用しない。
- batch 間で blind row を共有しない。
- zero-sum/zero-message subspace の sampling が偏っていない。

25.3 Fiat–Shamir

- transcript が prefix-free で canonical。
- protocol/version/circuit/field を domain separate。
- challenge を生成する前に対応する commitment root が hash へ入っている。
- challenge 再生成と proof parsing の順序が仕様通り。
- field reduction bias を処理。
- query indices が range 内で、duplicate rule が固定。
- proof malleability を防ぐため public input と statement digest を含む。

25.4 Merkle commitment

- leaf/internal hash domain separation。
- leaf serialization に length ambiguity がない。
- sibling order を含む。
- padding leaf が実 data として解釈されない。
- path pruning algorithm が root consistency を保つ。
- malformed path、過長 path、duplicate node を reject。
- hash output length が security target に適合。

25.5 proof parser

- 全 length を上限 checked integer で処理。
- field element が canonical range ℓ 。

- trailing bytes を reject。
- dimension multiplication overflow を防止。
- decompression bomb や巨大 allocation を防止。
- NaN/float を使わず整数 field arithmetic のみ。
- invalid curve 等はないが、invalid field modulus/roots を受け入れない。

25.6 side-channel

prover は witness を扱うため、field arithmetic、FFT access、sparse matrix traversal、hashing が秘密依存 branch を持たないことが望ましい。proof 自体は ZK でも、生成時間、memory access、failure log から witness 情報が漏れる可能性がある。

25.7 soundness regression test

- 一列だけ改ざん。
- 多数列改ざん。
- 正しい codeword だが linear constraint 違反。
- quadratic だけ違反。
- q の最高次係数だけ改ざん。
- blind row を zero に固定。
- duplicate query を悪用。
- Merkle root と leaf の mismatch。
- transcript challenge の round swap。
- batch instance の順序入れ替え。

統計 test では意図的に小 field を使い、観測 accept rate が理論 bound 以下か確認する。

26 性能評価の設計

26.1 測定すべき時間

- witness generation
- constraint compilation
- RS encoding / FFT
- response polynomial construction
- oracle transpose/store
- Merkle tree construction
- Fiat-Shamir / serialization
- query opening
- verifier circuit preprocessing
- Merkle verification
- algebraic checks

end-to-end と core protocol を両方報告する。

26.2 memory と I/O

peak RSS だけでなく、

- oracle raw size
- Merkle node size
- temporary FFT buffer
- disk spill bytes
- sequential/random I/O
- NUMA remote access

を測る。prover time が quasi-linear でも memory が線形以上であると、実用 scale では I/O が支配する。

26.3 proof size 内訳

$$|\pi| = |\text{roots}| + |\text{responses}| + |\text{opened columns}| + |\text{auth paths}| + |\text{metadata}|.$$

Ligero では回路が大きくなると opened columns が支配し、小さいと Merkle path と fixed response が支配する。この内訳を出さない proof size 比較は原因分析に使えない。

26.4 scaling plot

log-log plot で、

- slope $1/2$: proof size
- slope 1 または $1 \log N$: prover
- slope 1 : explicit-circuit verifier

を確認する。FFT size の 2 冪 rounding で sawtooth が出るため、理論 line と実測 point を分ける。

27 批判的評価

27.1 論文の最も重要な貢献

Ligero の歴史的意義は、単に「平方根 proof」を示したことではない。次の三点を同時に具体化したことにある。

1. MPC-to-ZK の抽象変換を、interleaved RS code の自己完結的 IPCP として書き下した。
2. transparent/hash-based proof が、巨大 PCP を経ずに実装可能であることを示した。
3. code distance、random linear compression、local opening の trade-off を parameter engineering へ落とした。

Aurora/STARK 以降の polylog proof が注目されても、Ligero の単純性と prover efficiency は独立の価値を持つ。

27.2 原論文の比較で注意すべき点

原論文は、改訂 Ligero が small-to-moderate circuit で後続 IOP 方式より短い proof と速い prover を示す。しかし、field size、statement model、machine、security calibration が異なる。したがって、これは「Ligero が特定 parameter 領域で Pareto frontier に残る」証拠であり、一般的勝者を意味しない。

27.3 方式上の限界

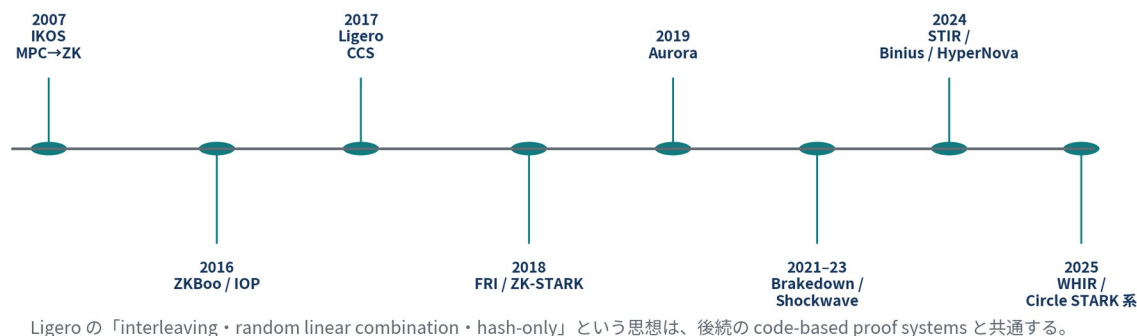
平方根 proof は、 N が極端に大きいと polylog proof に必ず負ける。verifier linearity は、proof を多数の light client や smart contract が検証する用途で致命的になり得る。さらに、現代 ZKP で重要な lookup、permutation、recursive aggregation を native に持たない。

27.4 それでも選ぶ理由

- structured setup を絶対に避けたい。
- DLP/pairing を避けたい。
- proof bandwidth より prover simplicity と time を重視。
- 同一回路 batch が大きい。
- security review で複雑な PCS/recursion stack を避けたい。
- educational/reference implementation として IOP の要点を明確にしたい。

28 研究ロードマップ

Ligero を中心とする透明 ZK の系譜



Ligero の「interleaving・random linear combination・hash-only」という思想は、後続の code-based proof systems と共通する。

図 18 MPC-in-the-head、Ligero、FRI/STARK、Aurora、linear-time code-based SNARK、近年の RS IOPP へ至る概略。

28.1 Ligero × modern IOPP

interleaved matrix の random row combination を、verifier が全 codeword membership check する代わりに、STIR/WHIR 型 IOPP で証明する。query 数を減らせるが、追加 round/oracle と proof fixed cost が増える。小中規模で Ligero、大規模で hybrid へ自動切替する設計が考えられる。

28.2 Ligero × folding

同一回路の instance を folding で accumulate し、最後の folded instance を Ligero で証明する。Nova 系の curve PCS を避ける hash-based folding が必要で、post-quantum 性と zero knowledge を保つのが課題である。

28.3 binary-field Ligero

Boolean witness を binary extension field へ直接埋め、XOR を free addition として扱う。additive FFT、tower field、bit-sliced implementation、binary-friendly hash を組み合わせれば、prime-field 版とは異なる hardware frontier が得られる。

28.4 list decoding と smaller fields

unique decoding $e < d/2$ を超え、list decoding 半径で random combination test を解析できれば、code rate をさらに上げられる。ただし extractor が一意 witness を得るには list ambiguity を別 challenge で解消する必要がある。

28.5 proof-carrying data

同一 relation の Ligero proof を単純に再帰検証するのは高価である。代わりに、oracle commitment の accumulation、correlated IOP batching、Merkle root aggregation を使い、複数 proof の query を共有する方向がある。

29 総括

Ligero の数学は、次の一連の圧縮として要約できる。

回路の全 gate 制約 witness の線形二次形式符号語行

→ ランダム行結合とランダム制約結合 → 少列の局所整合性 → Merkle + Fiat–Shamir argument.

この圧縮は、証明長を平方根まで下げる一方、prover を FFT・sparse linear algebra・hash という高速で監査しやすい primitive に保つ。Aurora は univariate sumcheck と FRI によって query を対数化し、STARK は AIR と FRI によって uniform computation の verifier を scalable にした。Groth16/PLONK は公開鍵代数と setup を受け入れて proof を極小化し、Nova 系は folding で逐次計算を扱う。

したがって Ligero は「古い STARK 以前の方式」ではなく、**透明性、post-quantum 志向、prover simplicity、proof bandwidth、verifier work の多目的最適化における明確な設計点**である。特に、同一回路の batch、十分大きい極端ではない回路、対称鍵 primitive のみを信頼したい環境では、現在も比較基準として重要である。

Appendix I 記号表

記号	意味
F	有限体
C	Boolean/算術回路
$s= C $	回路 gate 数
w	extended witness
m	witness matrix の行数
ℓ	witness matrix の message 列数
n	RS codeword 長
k	RS message dimension / degree bound
$d=n-k+1$	RS minimum distance
e	許容誤り列数の threshold
t	query 列数
σ	random compression 反復数
$\eta_1, \dots, \eta_n$	codeword evaluation points
$\zeta_1, \dots, \zeta_\ell$	message evaluation points
L	RS code $RS_{F,n,k,\eta}$
L^m	m -fold interleaved code
U	encoded witness matrix
$U[j]$	第 j 列 symbol
P_x, P_y, P_z	multiplication gate input/output selection matrix
P_{add}	addition constraint matrix
Q	random query set
r	random linear combination coefficients
u^0, u^{add}	blinding codeword rows
$q(X)$	linear constraint response polynomial
$p_0(X)$	quadratic constraint response polynomial
κ, λ	security/soundness parameter

Appendix II 定理依存関係

1. RS minimum distance $\Rightarrow$ 二符号語の一致点上限。
2. Lemma 4.2 / 4.5 / 4.6 $\Rightarrow$ random row combination が code 近傍へ落ちる確率。
3. Theorem 4.4 / 4.5 $\Rightarrow$ Test-Interleaved soundness。
4. random linear functional + polynomial root bound $\Rightarrow$ linear test soundness。
5. product degree bound $2k - 2 \Rightarrow$ quadratic test soundness。
6. 三 test の case split $\Rightarrow$ arithmetic-circuit IPCP soundness。
7. randomized encoding + affine blinding $\Rightarrow$ Lemma 4.15 perfect HVZK。
8. generalized affine tests + repetition $\Rightarrow$ Theorem 4.7 final ZKIPCP。
9. Merkle/commitment compiler $\Rightarrow$ interactive argument。
10. BCS/Fiat-Shamir + round-by-round soundness $\Rightarrow$ non-interactive ROM argument。

Appendix III 原論文図表との対応

原論文	内容	本書の対応
Figure 1	MPC から ZKIPCP 一般変換	図 2
Table 1	parameter 一覧	Appendix I で拡張
Figure 2	Test-Interleaved	図 5
Figure 3	Linear constraint test	図 6
Figure 4	Quadratic constraint test	図 7
Figure 5	generalized affine interleaved test	§ 7.9、Appendix B
Figure 6	generalized affine linear test	§ 7.8、Appendix B
Figure 7–11	parallel/affine generalized tests	§ 12.2
Table 2	proof length 比較	図 11
Table 3	prover/verifier time 比較	図 12・13

Appendix IV 用語集

affine blinding secret-dependent linear response へ、係数 1 で random codeword を加え、affine coset 上で一様化すること。

extended witness private input だけでなく、全 gate output、auxiliary bit、複製値を含む vector。

interleaved code 複数 codeword を行に並べ、列 vector を一 symbol とみなす code。

IPCP 最初に一つの PCP oracle を送り、その後短い interactive proof を行う model。

IOP 各 round で oracle message を送れる interactive oracle proof。

proximity word が code に属するかだけでなく、code から一定距離以上離れているかを区別する性質。

round-by-round soundness partial transcript ごとに good/bad state を定義し、次 challenge で bad から good へ移る確率を制御する soundness。

state-restoration soundness adversary が過去の verifier state から fresh randomness で分岐を試すことを許す soundness notion。

transparent setup secret trapdoor を含む structured parameter 生成を不要とし、公開 randomness または固定 hash 等だけを共有する setup。

主要参考文献

1. S. Ames, C. Hazay, Y. Ishai, M. Venkatasubramanian, “Ligero: Lightweight Sublinear Arguments Without a Trusted Setup,” CCS 2017; extended version, 2022.
2. Y. Ishai, E. Kushilevitz, R. Ostrovsky, A. Sahai, “Zero-Knowledge from Secure Multiparty Computation,” STOC 2007.
3. I. Damgård, Y. Ishai, “Scalable Secure Multiparty Computation,” CRYPTO 2006.
4. E. Ben-Sasson, A. Chiesa, N. Spooner, “Interactive Oracle Proofs,” TCC 2016.
5. E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev, “Scalable, Transparent, and Post-Quantum Secure Computational Integrity,” IACR ePrint 2018/046.
6. E. Ben-Sasson, A. Chiesa, M. Riabzev, N. Spooner, M. Virza, N. Ward, “Aurora: Transparent Succinct Arguments for R1CS,” EUROCRYPT 2019.
7. E. Ben-Sasson et al., “Fast Reed-Solomon Interactive Oracle Proofs of Proximity,” ICALP 2018.
8. J. Groth, “On the Size of Pairing-Based Non-interactive Arguments,” EUROCRYPT 2016.
9. A. Gabizon, Z. J. Williamson, O. Ciobotaru, “PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive Arguments of Knowledge,” IACR ePrint 2019/953.
10. A. Chiesa et al., “Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS,” EUROCRYPT 2020.
11. B. Bünz et al., “Bulletproofs: Short Proofs for Confidential Transactions and More,” IEEE S&P 2018.
12. S. Setty, “Spartan: Efficient and General-Purpose zkSNARKs Without Trusted Setup,” CRYPTO 2020.
13. A. Golovnev et al., “Brakedown: Linear-Time and Field-Agnostic SNARKs for R1CS,” CRYPTO 2023.
14. A. Kothapalli, S. Setty, I. Tzialla, “Nova: Recursive Zero-Knowledge Arguments from Folding Schemes,” CRYPTO 2022.
15. A. Kothapalli, S. Setty, “HyperNova: Recursive Arguments for Customizable Constraint Systems,” CRYPTO 2024.
16. B. Diamond, J. Posen, “Succinct Arguments over Towers of Binary Fields,” IACR ePrint 2023/1784.
17. N. Haböck et al., “STIR: Reed-Solomon Proximity Testing with Fewer Queries,” CRYPTO 2024.
18. N. Haböck et al., “WHIR: Reed-Solomon Proximity Testing and Polynomial Commitments with a Super-Fast Verifier,” EUROCRYPT 2025.
19. N. K. Nguyen, G. Seiler, “Practical Sublinear Proofs for R1CS from Lattices,” CRYPTO 2022 / IACR ePrint 2022/1048.
20. R. Canetti et al., “Fiat-Shamir: From Practice to Theory,” STOC 2019.